

Final Report Package

SP-5 Grocery Project (Blue)

CS 4850 – Section 02 – Spring 2026

5/2/2026

Sharon Perry

Team Members:

Jack Lusk

Jason Appolon

Amaya McCullough

Mahliq Obie

Links:

Website: <https://senior-project-blue-grocery.github.io/>

Github: <https://github.com/Senior-Project-Blue-Grocery/grocery-list-mobile-app>

STATS AND STATUS	
Lines of Code (LOC)	5969
Components/Tools	5: VsCode, Flutter, Dart, Firebase, Figma,
Hours Estimate	240
Hours Actual	212
Status	Project is 100% complete and working as designed

Table of Contents

Introduction.....	2
Challenges, Solutions to Challenges, Assumptions	3
Challenges	4
Solutions to Challenges.....	4
Assumptions	5
Requirements	6
Scope	7
Scope Statement	7
In-Scope	7
Out-Of-Scope	7
Constraints	7
Environment.....	8
System.....	8
Functional Requirements	8
Non-Functional Requirements	9
Additional Requirements	10
External Interface Requirements	11
User Interface Requirements	11
Hardware Interface Requirements.....	11
Software Interface Requirements.....	11
Communication Interface Requirements	12
Use Cases	12
Data Flow Requirements	13
Software Analysis and Design.....	14
Assumptions and Dependencies	15

Constraints	16
Implementation Approach	18
Software Architecture Strategy	18
Technology Stack Strategy	18
Modular Architecture Strategy	18
Data Persistence and Storage Strategy	19
User Interface Strategy	19
Reliability and Error Handling Strategy	19
Concurrency and Communication Strategy	19
System Architecture	20
System Architecture Drawing	21
Detailed System Design	21
Classification	21
Definition	21
Constraints	22
Preconditions and Postconditions	22
Component State Constraints	22
Data Format and Access Constraints	22
Synchronization Constraints	22
Timing Constraints	23
Exception Handling	23
Invariants	23
Resources	23
Interface/Exports	25
Development	26
Documentation - System and Integration Overview	26
Documentation - Account Management	28
Login Page	28
Username	28

Password	28
Register Account Page.....	29
User Account Storage.....	29
Documentation - Grocery Lists.....	29
Homepage	29
Create Grocery List	30
Delete Grocery List	30
List Items Subpage.....	30
Add/Remove Grocery Items	31
Real-Time Update Functionality	31
List Sharing Page.....	31
Documentation - Item Catalogue	32
Main Page	32
Search Functionality	33
Item Details Page	33
Item Categorization.....	34
Item Recommendation.....	34
Session Cart.....	34
Checkout Screen	35
Documentation - Item Database	35
Structure.....	35
Database Connection	36
Project Setup.....	37
Test Plan	38
Purpose	38
Content to be Validated.....	38
Scope of Testing.....	40
Test Cases.....	41
Test Procedures.....	45

Test Environment	55
Required Test Data.....	55
Test Report	56
Version Control.....	60
Summary.....	62
Appendix	63
Project Plan	63
Gantt Chart	65
Flutter Tutorial Completion	65
Figma Screen Mockups	66
Works Cited	66

Introduction

The SP-5 Grocery List App project is a mobile application that supports collaborative grocery list management using shared groups. The app allows users to create accounts, share accounts, and read, update, delete, and create grocery lists. As a key feature, the application is fully integrated with the cloud-hosted no-SQL online database Firebase. Firebase acts as back-end server infrastructure, allowing all updates to grocery lists to reflect in real-time, and for multiple users to share and update the same list. This enhances the shopping experience by allowing friends or family to update grocery lists wherever they are if they forgot an item or changed their preferences. The name chosen for this application is *Listin* - the electronic grocery list mobile app.

The key ethos that guided development was a “lightweight, community friendly grocery list manger that maximizes convenience”. In accordance, early development, including requirements specification and software design definition, focused on defining functional modules that encapsulate the key functionalities with the app, and modelling their connection with the online database. We also concepted a sleek, simple interface with an integrated item catalogue, which carried over to the final build. Development in accordance with the software design was carried out, resulting in the development and integration of six key modules: Account Management, Grocery List Management, Item Catalogue, Database Structure, User Interface, and Shared List Updates. The app was developed in the VSCode development and environment with support from Flutter and Dart. Thorough functionality and database testing was performed on all modules to ensure validity.

The goal of *Listin* is to enhance the grocery shopping experience by providing a convenient, clear, and collaborative platform to manage grocery lists alongside friends and family. This report encompasses the full scope of *Listin*'s development life cycle: providing detailed information and context on all stages of development.

Challenges, Solutions to Challenges, Assumptions

Challenges

- Challenge: Initial User Interface Design Conflict
 - There was some initial difficulty developing an effective model for the app's user interface, especially before the database connection was finalized. Some initial ideas that were pitched included a "one-screen" model where menus would dynamically move to show available items when a list was selected, but no ideas were sticking at first.
- Challenge: Database Structure Issues
 - When initially designing the database during early development phases, the structure of the database had to be redone multiple times. The substructure of JSON file database objects was difficult to determine before the modules were complete, but still necessary to facilitate developments. Most objects also used dummy IDs, so identifying them could be difficult. When the item catalogue was added, the Item Catalogue > Grocery Item database substructure had to be redone to include separate references to "purchase items".
- Challenge: Item Duplication Bug
 - A bug occurred in the middle stages of development regarding the item catalogue's search feature. When searching for a specific item, sometimes the same item would appear twice in the search results. The item would have the same name and identical information in all of its fields, but no associated picture.
- Challenge: Image Storage Issue
 - There were issues integrating the images for each grocery item. The nature of Firebase makes storing and later referencing grocery item images live somewhat difficult and unintuitive, compared to text formats. Additionally, acquiring the large number of pictures necessary was a challenge in itself.
- Challenge: Development Coordination
 - Since the two developers were specialized, one in front-end and one in back-end, there were some issues coordinating development. To be clear, both developers worked hard and were very cooperative. However, integrating the user interface

and other front-end module components with the back-end Firebase structure was time consuming and required redevelopment of some code.

Solutions to Challenges

- Solution: Initial User Interface Design Conflict
 - We met and performed a group brainstorming session to generate Figma interface mockups of all major screens. These served as a strong guide that solidified the development of the User Interface, flow between screens, and all widgets.
- Solution: Database Structure Issues
 - As the developers progressed with integrating the database, familiarity with how to use and structure the Firebase repository increased. By the time the session cart functionality was integrated, we created the finalized three-substructure database format, consisting of Users > UserID, Grocery Lists > ListID > Items > ItemID, and Item Catalogue > Item ID.
- Solution: Item Duplication Bug
 - Debugging revealed the cause to be an additional Firestone Service read() method call in the Item Catalogue search function when printing. This method call referenced the old Item Catalogue substructure which did not have integrated image URLs. Simply deleting this method call fixed the bug and stopped the item duplication.
- Solution: Image Storage Issue
 - The solution we arrived at for storing images was to use image URLs. Essentially, we stored the images in a separate Firebase repository created specifically to host images (Firebase has stricter rules regarding type definition). The images stored in this repository are referenced from the main repository. This is done using a field added to every itemID object, called the image URL, this is a direct reference to the image that can be loaded using a method call. So, an item's image can be loaded simply by calling the object and using its image URL as an argument to load the associated image.
- Solution: Development Coordination
 - This challenge was resolved by improving effective coordination between developers. As development continued, code familiarity and knowledge regarding widgets and Firestone methods improved. Additionally, the database structure being finalized made pulling it through the front-end with APIs much easier.

Regular code review and development coordination sessions were hosted, which defined goals for each developer and set the necessary level of collaboration.

Assumptions

The following are a variety of assumptions that were made during development. These assumptions guided development and molded the framework, priorities, and scope of the application.

- Database Integration would be the hardest aspect of development to integrate.
- Splitting the application into encapsulated modules would be the most efficient way to handle development.
- The UI and UX are high priority for a grocery list manager.
- The majority of users would prefer a simplified interface that minimizes the sequence of actions necessary to perform key functionalities.
- Item quantity is a necessary information field for grocery lists.
- Item Catalogue item instances must be stored separately from Grocery List item instances.
- The majority of users would not be particularly tech-savvy.
- Users would prefer to browse a detailed catalogue of pre-existing items than manually enter the names of items.
- Users would generally prefer having pictures of items.
- All database integration would be handled through Firebase repositories.
- All item object instances will pull live from Firebase with no exceptions.
- Shared List updates are seen immediately for all users in the group pipeline.
- Users can read English.
- Users have the basic know-how necessary to use a mobile application.
- Users own a functional smartphone with internet connectivity for online features.

Requirements

The following are the project requirements, which guided the development of *Listin*. These requirements reflect the final project build, and were conceived using the project statement and our own project plan. These project requirements were used to define *Listin*'s actual software design.

Scope

Scope Statement

Develop a mobile application for the creation and management of a grocery list. Users are able to create new grocery lists, add items to grocery lists, and remove items from grocery lists. The application includes a catalogue of grocery items to select from, including pictures to visually represent grocery items. The system will support linking additional users' accounts to a grocery list and integrate real-time synchronization across multiple accounts when a shared grocery list is updated. The project will also support user account creation and authentication, real-time online grocery list updates (data streaming), item adjustment features (quantity, priority, etc.), and a search feature for finding desired items. The application should feature an efficient and responsive front-end, with a robust backend that implements the list updating functionality between accounts and item price tracking. Active third-party grocery store data integration and item sale/discount integration are considered out of scope, but could be considered for phase 2 development.

In-Scope

Mobile-based platform, grocery list manager, grocery item catalogue, session cart, list creation, item search functionality, real-time online list updates, account linking, shared list updates, account creation and authentication.

Out-Of-Scope

Third-party grocery store data integration, item sale/discount integration, grocery item full nutritional information.

Constraints

Environment

The SP-5 Grocery List application is installed on a mobile platform and targeted at current generation smartphones. The system is based on Android and iOS platforms. Users should be in a position with a stable internet connection (Wi-Fi or cellular hotspot) to support real-time synchronization and shared list features. This application is considered for deployment on official application distribution platforms and will need standard mobile operating system permissions like network access and notifications.

System

The system is based on the client-server architecture. The mobile app is the client interface, and a backend service is used to authenticate users, store their data, and provide real time synchronization among the users. The back-end server is to contain user accounts, grocery items, carts, and item catalogue data that has been secured. The client and the server will communicate using standard network protocols (via Firestone APIs), so that the information can be integrated and maintained in all devices.

Functional Requirements

1. User Authentication and Account Management

1.1 Launch Page

The application shall display a launch screen containing the app logo and options to **Log In** or **Register**.

1.2 User Registration

Users shall be able to create an account using their first name, last name, email address, and password.

1.3 User Login

Users shall be able to log in using either a username or registered email address with a password.

1.4 Account Recovery

The system shall provide a password recovery feature that sends reset instructions to the user's registered email.

1.5 Profile Management

Users may enter a valid email address for their account to send and receive invitations to share grocery lists with other users.

2. Grocery List Management

2.1 Item Browsing

The application shall provide a scrollable menu of grocery items with images that users can browse and add to their list.

2.2 Search and Sorting

Users shall be able to search for specific items and sort available items within the menu.

2.3 Item Categorization

Items shall be organized into categories such as produce, dairy, meat, and pantry.

2.4 List Editing

Users shall be able to edit their grocery list by changing item quantities or removing items.

3. Collaboration and Social Features

3.1 Shared Lists

Users shall be able to link accounts with other registered users to create shared grocery lists.

3.2 Editing Permissions

All members of a shared list shall be able to add, edit, and delete items on that list.

3.3 Real-Time Synchronization

Updates made by one user shall automatically synchronize across all linked devices.

3.4 Group Notifications

The system shall notify group members when items are added, edited, or removed from a shared list.

4. User Interface and Smart Features

4.1 Settings Menu

A settings tab shall be available for managing user preferences and logging out.

4.2 Screen Navigation

The system shall provide an interface for navigating between all available screens which achieve core functionalities.

4.3 Recommendations

The system should recommend frequently purchased or discounted items based on user behavior.

Non-Functional Requirements

1. Security

- 1.1. Users must authenticate using their secure login credentials (username/email + password).
- 1.2. All data transmission between the mobile app and server must use HTTPS/SSL encryption.
- 1.3. Only members of a grocery group can view or modify that group's list (group-based access control).
- 1.4. The system must prevent unauthorized database access and protect against common threats such as SQL injection.
- 1.5. User sessions and database connections should automatically expire after logging out to prevent simultaneous access to the same account.

2. Capacity

- 2.1. The app must support multiple concurrent users modifying lists at the same time.
- 2.2. Real-time updates should propagate to all connected users with minimal delay (e.g., within 1–2 seconds).
- 2.3. The database should support storing data for thousands of users and grocery items.
- 2.4. The system should scale to handle increased usage without performance degradation.
- 2.5. Each grocery list should support a maximum reasonable number of items (e.g., 100+).

3. Usability

- 3.1. The interface should be simple and intuitive for users of all technical skill levels.
- 3.2. Common tasks (add item, edit item, delete item) should require minimal steps.
- 3.3. The app should provide clear feedback when actions succeed or fail.
- 3.4. The design should follow standard mobile UI guidelines (buttons, lists, icons).
- 3.5. The app should load quickly and respond smoothly to user input.

Additional Requirements

Reliability

- The system should maintain data integrity even if a device loses connection.
- No data should be lost during synchronization.

Performance

- List updates should be processed in real time with minimal lag.

- The app should launch within a few seconds.

Compatibility

- The app should run on Android and IOS mobile platforms.

External Interface Requirements

User Interface Requirements

- The application shall provide an intuitive and user-friendly interface suitable for mobile devices.
- The user interface shall include clearly labeled navigation elements such as tabs or menus.
- Grocery items shall be displayed with images and readable text.
- The interface shall support both light mode and dark mode.
- Buttons and interactive elements shall be large enough to accommodate touch input.
- Error messages and notifications shall be clear and informative.
- The interface shall update dynamically to reflect real-time changes made by shared users.

Hardware Interface Requirements

- Processor with minimum 2 GHz speed + 64-bit architecture.
- 2-3 GB of RAM.
- Adequate phone storage / disk space of 300 MB.
- Minimum screen resolution to properly render graphics / UI.
- Functional touch screen.

Software Interface Requirements

- **Mobile OS Interface:**
 - o The application must run on modern mobile operating systems (e.g., Android and/or iOS) and use standard OS services for networking, storage, and notifications.
- **Backend API Interface (Client–Server):**
 - o The mobile client must communicate with the backend server through a secure web API.
 - o The API must support operations for user authentication, group management, and grocery list CRUD (create/read/update/delete).
 - o Data must be exchanged using a standard format such as **JSON**.

- **Real-Time Synchronization Interface:**
 - o The system must support real-time updates between devices using a real-time database or messaging method (e.g., real-time listeners, WebSockets, or a real-time backend service).
 - o When one user updates a list item, all other group members' devices must receive the change immediately.
- **Database Interface:**
 - o The backend must interface with a database system to store:
 - User accounts and credentials (securely).
 - Group membership and access permissions.
 - Grocery lists and list items.
 - The database must support concurrent read/write operations safely.
- **Authentication/Authorization Interface:**
 - o The backend must enforce authentication and group-based authorization.
 - o Only authenticated users can access the system.

Communication Interface Requirements

- Functional Wireless Network Interface Controller to access online functionality.
- Http package for networking and network requests.
- BaaS / custom backend for communication between accounts (handeld through Firestone Services).
- FlutterUI.
- WebSockets.

Use Cases

Use Case 1: User Registration

Actor: New User

Description: User makes a new account by entering the necessary personal data.

Result: The creation of a new user account followed by its storage in the system is successful.

Use Case 2: User Login

Actor: Registered User

Description: This involves the user entering the application with valid credentials.

Result: The user accesses his/her available shared and shopping lists.

Use Case 3: Create Grocery List

Actor: Authenticated User

Description: The user can make a new grocery list.

Result: New grocery list is stored and shown in the account of the user.

Use Case 4: Edit Grocery List

Actor: Authenticated User

Description: The user adds or deletes items in a grocery list.

Result: The grocery list is updated in real-time.

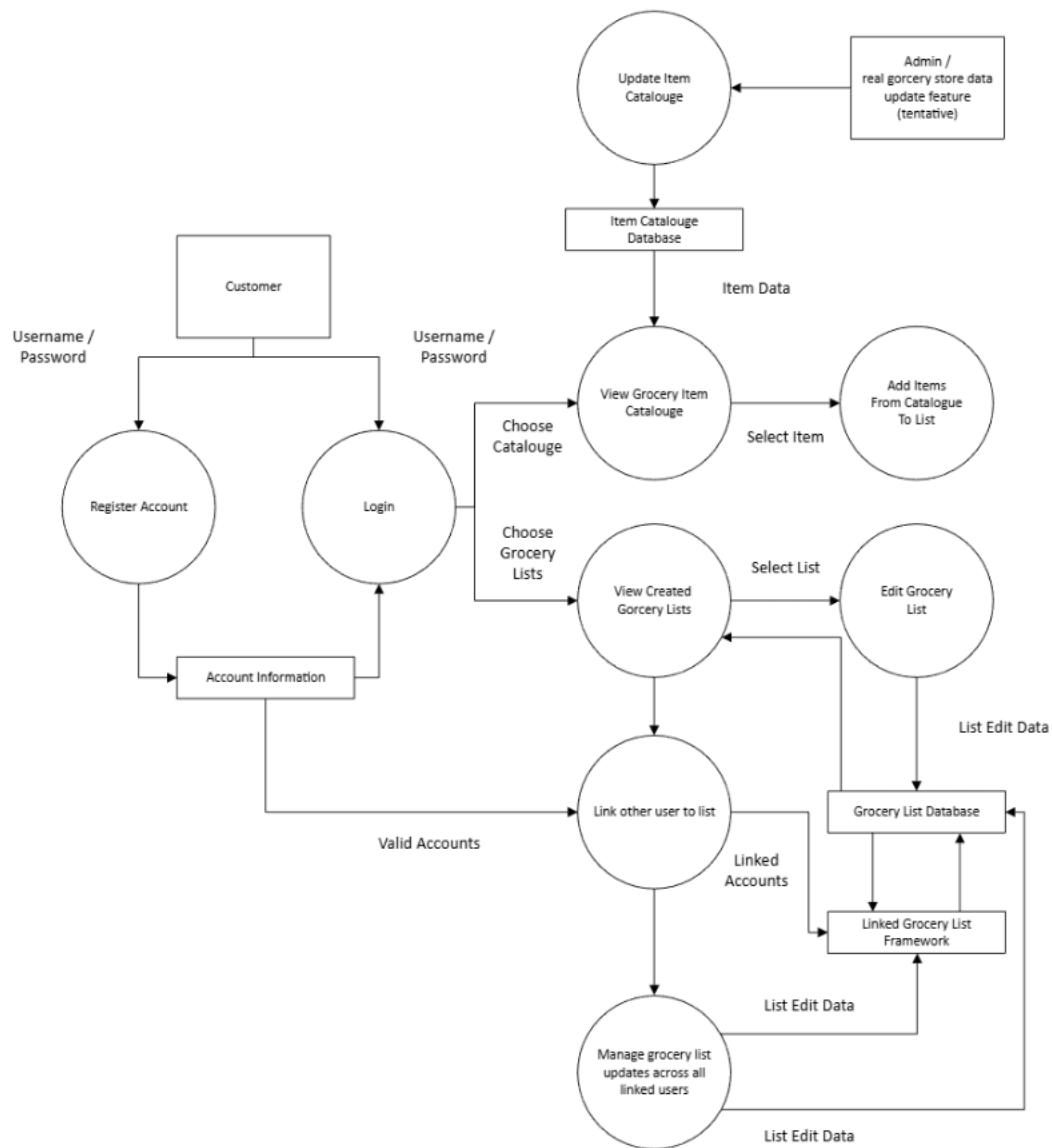
Use Case 5: Share Grocery List

Actor: Authenticated User

Description: The user provides another registered user with a grocery list.

Result: The shared list can be jointly edited by both users.

Data Flow Requirements



Software Analysis and Design

Listin is a mobile application that supports collaborative grocery list management using shared groups. In requirements specification, the following application features were defined:

Listin allows users to create accounts, share accounts, and individually read, update, delete, and create grocery lists. Users can simultaneously alter the contents of multiple grocery lists to their likings and see changes in real time.

These requirements were analyzed to define the models, environment, and methodology used to develop the practical software implementation of the grocery list mobile application.

Assumptions and Dependencies

- Hardware System Assumptions and Dependencies
 - o The application will run on smartphones/tablets.
 - o The application's fairly lightweight nature leads to the assumption that the majority of modern mobile devices will have sufficient processing ability and storage to run the application without notable issue.
 - o Key mobile device functionality such as touch input (scrolling, tapping) and local app storage are necessary to use the application.
 - o Internet functionality and a functioning internet connection are necessary for the user linking feature to function.
- Software System Assumptions and Dependencies
 - o Application is compatible with the following mobile OS versions: Android 8.0 or Android 9.0 and IOS 15 or later to ensure general functional compatibility within the standards of the modern mobile user base.
 - o Development via the cross-platform framework flutter will allow for separate, native deployments of a shared codebase across different mobile OS (Android and IOS).
 - o The application assumes that the user is running the application on a supported mobile Operating System.
 - o The application may rely on the following additional software services or functionalities for various reasons:
 - Utilizing common mobile OS features such as push notifications
 - Cloud storage or server-based data storage services for synchronizing lists between multiple accounts and devices.
 - Real-time remote data synchronization software protocols for the implementation of the shared list updating feature.
 - Security and protection for the application.

- o Updates or changes to third-party libraries or mobile OS could require software updates or bug fixes
- End-User Characteristics
 - o Target age demographic is fairly wide. Users between the ages of 16 to 60, who regularly shop at grocery stores are expected to be the primary user-base of the app.
 - o The user base assumes a large range in terms of age and technical skill due to the simplicity and intuitiveness of the interface and application functionality.
 - o Familial units with multiple able shoppers may be considered a favored target user base due to the application's grocery list sharing and real-time synchronization.
 - o Users are not expected to have significant experience navigating complex applications or using complex interfaces.
 - o Particularly young users (ages 10-16) are not expected to use the application.
 - o The application is designed for an English-speaking user base and will not feature additional language options.
- Probable Changes in Functionality
 - o The system is expected to evolve, with additional features that enhance functionality potentially being added during phase-two of development. The most likely features are:
 - Monetize the app by introducing features such as direct user payment, monetary authorization, or a user subscription service. Integrate a secure payment gateway.
 - Integrate databases such as the grocery item catalogue with third-party data from legitimate sources such as grocery stores.
 - o Scalability changes are made as necessary to ensure the application can support a requisite number of users based on derived project goals.
 - o The level of granularity for searching functionalities, such as item catalogue searching or user searching, is expected to change based on feedback,

Constraints

- Hardware and Software Environment
 - o The system must operate on modern smartphones using Android and iOS operating systems. Because mobile devices have limited processing power, storage, and battery life compared to desktop computers, the application must be lightweight and efficient. This constraint requires the use of a cross-platform mobile framework and optimized code to ensure the app runs smoothly across different device models and screen sizes.
- End-User Environment

- Users will interact with the system using touch-screen mobile devices and varying levels of technical experience. Therefore, the interface must be simple, intuitive, and require minimal steps to perform common actions such as adding or editing grocery items. The design must also support different screen resolutions and orientations.
- Network Communication
 - The application depends on an active internet connection to support account authentication and real-time synchronization of shared grocery lists. Since users may have unstable Wi-Fi or cellular connections, the system must handle temporary disconnections and automatically resynchronize data when connectivity is restored. This constraint requires the use of a client-server architecture and background data synchronization mechanisms.
- Data Storage and Distribution:
 - All user accounts and grocery list data must be stored in a centralized backend database rather than locally on the device. This requirement ensures shared lists update across multiple users, but it also impacts the system design by requiring database management, remote data access, and safe concurrent read/write operations.
- Security Requirements:
 - The system must protect user information and shared list data. User authentication, encrypted communication (HTTPS), and access control are required, so only authorized group members can view or modify a grocery list. These requirements affect the software design by requiring secure login handling, protected database storage, and secure server communication.
- Performance and Capacity Limitations
 - The application must support multiple users updating the same grocery list at the same time. Real-time updates should appear with minimal delay. This requires efficient server processing, optimized database queries, and lightweight data transmission formats.
- Interoperability and Interface Requirements
 - The mobile application must communicate with a backend server using a web API. Data must be transmitted in a standard structured format such as JSON. This constraint affects system design by requiring defined request/response messages, standardized endpoints, and consistent communication protocols between client and server.
- Testing and Validation
 - The system must be tested across different devices, operating system versions, and network conditions to ensure reliability. Because real-time synchronization is a core feature, testing must include multiple users interacting with the same list

simultaneously. This requirement impacts development by requiring additional validation procedures and simulated multi-user testing.

Implementation Approach

The application development is conducted using an object-oriented and iterative prototyping approach. Object-oriented design is used to model core entities such as grocery items, categories, and lists as reusable classes, improving code organization, maintainability, and scalability. This approach allows functionality such as adding, editing, and removing items to be encapsulated within well-defined components. UML-style modeling was used during the design phase to plan class structures and interactions between components.

Iterative development and prototyping methodology will also be used. Initial versions of the application will focus on basic functionality, such as displaying a catalog of grocery items and supporting user input. Additional features and interface refinements are defined based on testing and usability considerations. This approach enables early detection of design issues and allows improvement to be made without large refactoring.

Flutter, a component-based UI framework, supports reactive programming and separation of concerns between the user interface and application logic. State management techniques are used to ensure the UI updates dynamically in response to user actions, such as adding items to a list or checking an item off.

Software Architecture Strategy

Technology Stack Strategy

The system is built based on Flutter framework using the Dart programming language. Flutter enables a common codebase to be applied on Android and iOS platforms, saving on development time whilst providing close to native performance. Other solutions like developing discrete native apps (Java/Kotlin on Android and Swift on iOS) were also looked into, however, dismissed because of higher development and maintenance cost. Flutter was selected due to its looseness, performance, and extensive library ecosystem.

Modular Architecture Strategy

The architecture of the system employs a modular design whereby the functionality is broken into layers including:

- User Interface (UI) Layer

- Business Logic Layer
- Data Management Layer
- Network/Service Layer

This division enhances maintainability, readability, and testing. The modules have a certain role to perform and can be easily expanded to add features in the future without affecting the irrelevant parts. We are not considering a monolithic design since it is more challenging to debug and scale as the application grows.

Data Persistence and Storage Strategy

The application will use local storage and remote cloud service.

- Caching of user preferences and session data is done in the local storage.
- Persistent data (user accounts, grocery lists, and shared items) are stored remotely.

This mixed strategy enables the application to be effective and at the same time ensures that data is maintained for all devices. Full local storage was eliminated because it would not allow an option of multi-device syncing and collaboration.

User Interface Strategy

The UI is based on the mobile-first interaction model, which is touch-based and uses Flutter widgets. The screens are made simple and responsive so as to be usable irrespective of their size. The control of navigation is supported using structured routes and state management in order to allow users to easily travel among views including login, item listing, and editing.

Reliability and Error Handling Strategy

The system includes detailed error management protocols to handle problems like network failures, bad inputs and services that are not available.

- Raw errors are not shown but handled through user-friendly messages.
- Debugging and maintenance are done through logging.
- Fallback behavior is put in place to handle temporary network problems.
- Failed database object pulls will not cause a crash and will instead display an empty placeholder object and provide a warning.
- Sessions are immediately exited, and the application closed if an exception-related crash occurs to prevent account mismanagement.

This makes the system usable in imperfect conditions.

Concurrency and Communication Strategy

The network requests and local processing activities are addressed asynchronously so that the UI is not frozen. The use of the Flutter model, which is an asynchronous programming model utilizing Dart's single-threaded event loop, enables the system to process many tasks including data synchronization, loading lists, and updating a screen at the same time by permitting non-blocking operations. This enhances user performance and responsiveness.

System Architecture

The major functionalities and subsystems were partitioned and assigned to components in the following manner. The system utilizes a client-server architecture, dividing the system into the client-side, backend services, and real-time data synchronization, and data storage subsystems. We believe this division of roles cleanly partitions the key functionalities of the system. The subsystems are clearly delineated and interact smoothly in expected manners while ensuring key functionalities like real-time synchronization.

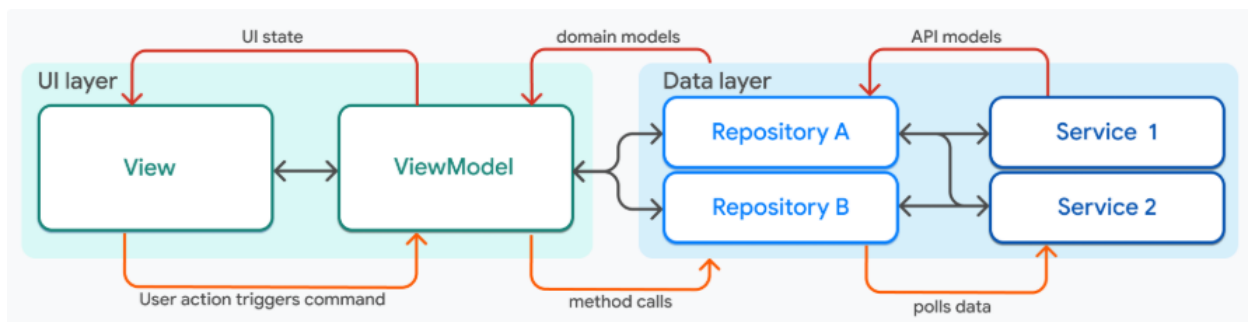
The function of the client-side subsystem is to handle front-end user interaction, and interact with the backend service as necessary, receiving data and submitting changes to achieve the functionality of an interactive grocery application. This front-end maintains local visual representations of the grocery lists and the item catalogue, and communicates user actions towards these data structures to the backend. The client-side front-end is developed using Flutter and Dart, particularly utilizing the Flutter SDK framework for UI development.

The function of the backend services subsystem is to handle core application logic. Notably, the creation of lists, the editing of lists, the addition and removal of catalogue items to the list, user-to-user list linking, permission enforcement, and querying. Centralizing these responsibilities here ensures that changes to the shared lists and item catalogue data structures are applied consistently across multiple users. Backend services are handled using the Dart Frog server-side framework for the Dart language. Additionally, RESTful API with standard HTTP protocols is used for interaction.

To support responsiveness across linked user grocery lists, the system should feature a real-time synchronization subsystem that propagates updates made to a linked grocery list across all linked clients. To ensure that users see the updated list state with minimal latency, this subsystem will interact with the front-end and backend services as necessary. It has been separated from the other backend services due to its nature as a critical and fairly complex functionality.

The data storage subsystem is responsible for maintaining the structure and consistency of all application data, including user account data, item catalogue data, grocery list data, list sharing relationships, etc. It should be designed to support concurrent updates and maintain consistency across multiple users editing the same list. By isolating data storage and structure concerns, the system can enhance its storage strategy (scaling data, replicating data, third party data integration) without impacting client behavior or application logic. SQL relational databases or cloud storage are used to store and persistently structure data.

System Architecture Drawing



Detailed System Design

Classification

The main components of the system are classified as follows:

- **Subsystems** – Major functional parts of the application (UI, Data, Network).
- **Modules** – Groups of related features (authentication module, list management module).
- **Classes** – Objects representing users, items, and lists.
- **Services** – Components responsible for communication with external systems.
- **Functions** – Individual operations such as addItem(), removeItem(), or syncData().

Each classification helps organize the system and define responsibilities clearly.

Definition

The elements of the system are defined as follows:

- UI Components are used to interact with the user and render screens.
- Authentication Components control user registration, user login and user session.
- List Management Components deal with the creation, editing, deleting and distributing of grocery lists.
- Data Storage Components handle the storage and retrieval of data both at the local level and remotely.
- Network Components handle communication with APIs and cloud services.

These elements combined contribute to the main functionality of the application and establish data consistency, performance, and usability.

Constraints

Preconditions and Postconditions

Preconditions

Certain conditions must be satisfied before the component can function properly:

- The user must be authenticated and logged into a valid account.
- The user must belong to an existing grocery group or create a new list.
- The device must have a network connection in order to synchronize with the server.
- The requested grocery list must exist in the database, and the user must have permission to access it.

If these conditions are not met, the system will deny access and display an appropriate error message.

Postconditions

After successful execution of a grocery list operation (add, edit, or delete item), the following must occur:

- The modified data is saved to the backend database.
- The server confirms the update.
- The updated list is transmitted to all connected group members.
- The user interface refreshes to display the most current list of contents.

Component State Constraints

Each component maintains consistent list data across devices. At any time, the grocery list must reflect the most recent confirmed server version. Local device copies of the list are considered temporary and may be replaced by synchronized server data. The system must not allow unauthorized users to modify or view a list. If a user leaves a group, the list data must no longer be accessible to that user.

Data Format and Access Constraints

- Grocery list data is exchanged between the client and server using structured data (JSON format).
- Each item must contain required fields such as item name and quantity.
- Invalid or empty item values are rejected by the system.
- Only authorized users can perform create, read, update, or delete (CRUD) operations.

Synchronization Constraints

Because multiple users may edit a list simultaneously, the component must handle concurrent operations:

- Updates are processed by the server rather than locally on the device.
- If two users edit the same item at the same time, the most recent confirmed update is stored and distributed.
- The application automatically refreshes the list after synchronization.

Timing Constraints

- Real-time updates should appear on other users' devices within a short delay (approximately 1–2 seconds).
- The application should not freeze while waiting for server responses.
- If the server does not respond within a timeout period, the request will fail and the user is notified.

Exception Handling

The component must handle error situations gracefully:

- If the network connection is lost, changes are temporarily stored and synchronized when the connection returns.
- If authentication fails, the user is redirected to the login screen.
- If the server rejects invalid data, the application prompts the user to correct the input.

Invariants

The following conditions must always remain true:

- Every grocery list belongs to a valid group.
- Only group members can view or edit that list.
- The database copy of the list is the authoritative version.
- Data integrity must be preserved during synchronization.

Resources

The SP-5 Grocery List application depends on several external system resources to function properly. These resources are managed by the software but exist outside of the application code itself.

- Mobile Device Resources
 - The application runs on a user's smartphone and requires access to device hardware resources including the processor, memory (RAM), storage, and network interface. The processor is used to render the user interface and process user interactions such as adding or editing items. Memory is used to temporarily store active grocery lists and session information while the app is running.

Limited mobile memory requires the application to minimize background processes and clear unused data when not needed.

- Network Resources
 - The system requires an internet connection (Wi-Fi or cellular data) to authenticate users and synchronize grocery lists between multiple devices. The application communicates with a remote server using secure network requests. If connectivity is lost, the app must temporarily cache user actions locally and update the server once the connection is restored.
- Database Resources
 - A centralized backend database is required to store user accounts, group memberships, and grocery list items. The database allows multiple users to access and modify the same shared list. Because this is a shared resource, it must support concurrent read and write operations without corrupting data.
- Backend Server
 - A backend server is responsible for handling authentication, processing requests, and distributing real-time updates. The server manages communication between all connected mobile clients and ensures changes made by one user are transmitted to other users in the same group.
- Software Libraries and Frameworks
 - The system depends on external software frameworks such as a cross-platform mobile framework (e.g., Flutter or React Native) and networking libraries used to communicate with the backend server. These libraries provide UI rendering, data handling, and secure communication capabilities.
- Race Conditions
 - A race condition may occur when multiple users attempt to edit the same grocery list item at the same time. For example, two users could change the quantity of “milk” simultaneously, causing conflicting updates. To resolve this, the server will process updates sequentially and use timestamps or last-write-wins conflict resolution to ensure the most recent change is stored. The system may also refresh the list automatically, so all users see the updated value.
- Deadlock Situations
 - Deadlock could occur if multiple processes attempt to lock the same data simultaneously and wait on each other indefinitely. Although mobile applications rarely lock entire databases, simultaneous updates to shared list data could cause waiting conditions on the server. This is prevented by short-duration database transactions and automatic request timeouts, so the server releases locked resources quickly and continues processing other requests.

Interface/Exports

User Interface Component

Classification: Subsystem

Definition: Offers displays and controls to enable users to communicate with the application.

Exports / Services:

- Show login and home screens.
- Browse (lists, items, settings).
- Receive user input and provide system feedback or error messages.

This component links the system's functionality to the user.

Authentication Component

Classification: Module

Definition: Controls access to the application by a user.

Exports / Services:

- Enable the users to create accounts and log in.
- Monitor user session status.
- Check out and authorization.

This component ensures that the system is used by authorized users.

List Management Component

Classification: Module

Definition: Deals with the creation of grocery lists and editing grocery lists.

Exports / Services:

- Create and delete lists.
- Add, update, and remove items.
- Shares support with other users.

This component allows users to control their list data effectively.

Data Storage Component

Classification: Service

Definition Stores and retrieves application data.

Exports / Services:

- Store user and list data on the disk.
- Retrieve saved information.
- Accommodate remote storage, whenever possible.

This component guarantees data persistence between sessions.

Network Component

Classification: Service

Definition: Handles communication with external services.

Exports / Services:

- Transmit and get data to servers.
- Identify and manage connectivity issues.
- Servers: Fraud Support Data synchronization.

This component enables the system to operate in interconnected environments.

Development

Provided is the development documentation, which records the overview of all finalized software component functionalities and how they were implemented in the development phase. This is a running summation of development activity.

Documentation - System and Integration Overview

- **Integration Process:** Overall development followed a traditional Component-Based Software Development approach. Individual software components were designed largely independently via encapsulation principles, then were added to the integrated Flutter development environment via VScode. This allows for modular modification of different components, which is useful for testing and manipulating app features, like adding items. The integrated cloud database, Firebase, acted as a framework which tethered the backends of every module, as all components interact with it in some way.
 - o Application components, or modules, are encapsulated using a Flutter “screen”, and its associated widgets, functions, and backend. The current major modules in this application are Account Management (Login Screen), Grocery Lists, Item Catalogue, and the Item Database (backend). The application runs one screen at a

time, and user interaction to leave a module is handled by a function call to perform a screen switch, leaving the current module and running the new module. Actual permanent grocery list changes are handled by backend database writes. These updates are reflected across all modules immediately by StreamUpdate().

- **Operational Workflow:** Currently linear, though nonlinear access to the item catalogue is being considered for later development. A single run consists of the user:
 - o Running the app > Open to login page > Touch register account > Enter account information via database write > Return to login page > Login > Open to Grocery List Management Page > Touch Add List > Name List > List is added to management page via database write > Touch List > Open to Individual List Page > Touch add items > Open to Item Catalogue > Select desired list items > Move to Individual Item Pages > Select desired item quantity > Touch the checkout button > Move to Session Cart > Confirm Items > Write new items to chosen list > Return to Grocery List Management Page (now displays new items) > Close App
 - o Module Flow is: Account Management > Grocery Lists > Item Catalogue, with all modules accessing the Item Database.
 - o Cross-Account Shared List Updates have been integrated as a component of the Grocery Lists module, and the functionality may be accessed from another module at any time.
- **Global Design Decisions:**
 - o As the Firebase cloud database is used for the backend framework, all modules integrate Firebase libraries and instance objects, and general application logic is often handled by instance reads and writes to the database.
 - o Modules with visual UI run StreamUpdate() to show grocery list and item updates in real-time to the user as quickly as possible.
 - o JSON was selected as the database file format for its standardization and ease of use.
 - o Flutter widgets with associated functions were chosen to handle all interactive features for screen builds.
 - o General UI, UX, and color design were chosen in advance based on Figma mockups. UI is consistent throughout the application.

Documentation - Account Management

Login Page

- **Conceptual Overview:** Traditional login screen UI. Provides fields for a username and a password, and an option to register your account. First application screen, and the first thing the user sees on a successful run.
- **Development Process:** Traditional screen build that runs from main on startup. Fields for username and password, and related buttons are handled by widgets. Utilizes the traditional minimalist blue and white visual design proposed in mockups. Uses a firebase authentication instance to log the user in when they type in their username and password. This connects the user to the database and logs them in. User is taken to the Grocery Lists module homepage upon successful login.

Username

- **Conceptual Overview:** Username is a valid email address provided by the user upon successful account registry in the register account page. Only valid emails may act as usernames. Successful entry of the username on the Login Page will move the user to the Grocery Lists homepage.
- **Development Process:** Username is a character string of variable size that is written to the specified Firebase userID document upon successful account registry via instance object. The current build does not implement any email-related username limitation functionality. Any username within the character limit is valid.

Password

- **Conceptual Overview:** Password acts as the key functionality for the identification and authentication of user accounts. The creation of a password is done through the register account screen interface. Attempts to register an account password will fail unless a password complies with the specified security policy
- **Development Process:** Passwords are written to the specified userID document in Firebase upon successful registry in the same manner as usernames. For a password registry to be valid, it must meet the following checks:
 - The password is a minimum of 8 characters long.

- o The password contains at least 1 uppercase alphabetic character.
- o The password contains at least one lowercase alphabetic character.
- o The password contains at least one numeric character.
- o The password contains at least one special character

Register Account Page

- **Conceptual Overview:** Accessed from the Login Page via a “Register Account” button. Takes a user provided username and password to create an account. Then the user is moved back to the login screen upon successful account creation. The code register interfaces with Firebase authentication to create the account, which is then saved to a document in the database. Username and password must meet certain specifications.
- **Development Process:** Uses two TextField widgets to get desired username and Password from user. Upon selecting “register”, specified sequential logical checks are performed on the username and password to check validity and redundancy. If a single check fails, both text fields reset and an error message is printed to screen. If all checks succeed, a Firebase instance object is called, which creates a new UserID document and writes the username and password to it. The user is then automatically returned to the Login Page.

User Account Storage

- users (collection) - list of all users
 - └─ userId (document = Firebase UID) - each user is an individual document

username: john@email.com

password: Gr0c@ryFan

Documentation - Grocery Lists

Homepage

- **Conceptual Overview:** This is the User Interface the user accesses immediately after a successful login. It acts as the central home screen for the application. It displays a visual representation of the names of all the grocery lists the user has created. On touch interaction with a created list, the user is taken to a separate page which contains the item information of that particular list.

- **Development Process:** User lists are displayed by referencing all lists associated with the user in the database through a method call and displaying the information visually. All lists are displayed in a column format with individual lists appearing as cells, or rectangles. A function call, with the assistance of `StreamUpdate()`, chooses the number of rectangles to draw by referencing the database, allowing for updates to occur immediately in real-time. Lists are obtained by referencing the grocery list Firebase collection and reading in all lists with an `OwnerID` value that matches the current `userID` from this login instance.

Create Grocery List

- **Conceptual Overview:** User enters the name of the new desired grocery lists and interacts with an add list button. This inserts a new empty list into the database and immediately displays it on the Grocery List homepage.
- **Development Process:** This functionality (creating a new grocery list) was achieved through an interactive button widget accessible on the main page. This calls for the add list function, which generates a new list and inserts it into the database. Notably, it generates a document with a unique `listID` and inserts it into the `groceryLists` collection to accomplish this. The item subcollection starts as empty for all newly created lists.

Delete Grocery List

- **Conceptual Overview:** User selects the grocery list they want to delete, and it is removed from the database. The list is then removed from the Grocery List homepage.
- **Development Process:** This functionality (deleting a current grocery list) was achieved through an interactive button widget accessible on the main page. This calls for the delete list function, which deletes a specified list document by removing it into the database. This delete cascades to all item documents in the list's subcollection. This function is activated through touch interaction with a red trash can widget graphic to the right of each individual list.

List Items Subpage

- **Conceptual Overview:** This page is accessed by touch interaction with a created list. This takes the user to a separate page that displays all of the items, and their associated quantities and prices, for the particular list.
- **Development Process:** This screen is built upon interaction with a created list cell in the homepage, and is a separate instance from it. The item subpage uses the same structure and formatting as the grocery list homepage, just displaying individual list items instead of full lists. This information is obtained by referencing the items subcollection documents for that particular list in the database. Items, which are represented using

documents in the database, have their name, quantity, and price read when item cells are drawn. By default, this page will contain no items for a newly created list. By selecting the add items button, the user is linked to the item catalogue page to select items for the list.

Add/Remove Grocery Items

- **Conceptual Overview:** The adding and deleting of grocery list items is handled via standard methods. The add items method is used to add a specified item to a particular grocery list. This method references the specified item and appends it to the associated grocery list via the items collection in the database. The remove items method functions similarly, referencing the specified item and deleting it from the associated grocery list document.
- **Development Process:** Development implementation has differed significantly from the original concept. While delete is still achieved through a standard deleteItem() function, which removes the specified item document from the item subcollection upon interaction with a trash can button widget, adding grocery list items is handled differently. AddItem() is handled exclusively by the item catalogue module. Writes to the items collection are done in bulk based on a “shopping cart” structure of specified list items (Select list > Select add Item > Move to Item Catalogue > Choose Items > Add to list).

Real-Time Update Functionality

- **Conceptual Overview:** Updates to key data structures, including grocery lists and item sub-collections, must occur and visually reflect across all UI object representations immediately in real-time. Operations that must implement real-time updates include adding a grocery list, deleting a grocery list, adding list items, and deleting a list item.
- **Development Process:** Updates to grocery list items via the add or remove items/grocery list functionalities are handled using a stream builder widget, which displays updated information based on the latest snapshot. StreamBuilder() acts as a listener which automatically rebuilds the UI when new data enters from the stream (captured in snapshots).

List Sharing Page

- **Conceptual Overview:** The list sharing page is a separate screen accessed from the central navigation interface that encapsulates all aspects of account linking to modify shared grocery lists (encompassing both sending and receiving). Users may send an invitation to share a specified list to another user, who may then accept the invitation from their own list sharing page. If the invitation is accepted, the list will appear in the

accepting user's grocery lists main page, and all updates to the list are reflected for both users.

- **Development Process:** Development implementation of this module was one of the last completed aspects of the project, so the database structure and other modules were finalized by this point. This page may be accessed from the bottom screen navigation interface. Upon loading the screen, the user's currently created grocery lists are pulled. The user may select *send invite*, and a sub-menu will open where the user can select an available list and enter an email address to select a recipient. If the email address matches a valid user account, the invite is sent to that account's account linking page. This is done by performing a write to the User Account structure in Firebase, which enters the invite information as a field in the specified account substructure. An account can host multiple invites on the account linking page, and all invites can either be accepted or refused. Accepting an invite adds that list to the account's Grocery List homepage. This is done by assigning all invited users besides the original user a pointer variable which references the original grocery list structure in database memory. Now, all item additions and deletions to the list will affect the common shared list, and be reflected in real-time for all accounts through StreamBuilder() updates. There are no limits on the number of users who can share a list.

Documentation - Item Catalogue

Main Page

- **Conceptual Overview:** The item catalogue visual user interface is created using Flutter robust User Interface / widget tools. The Item Catalogue is designed with a sleek, professional aesthetic, and is meant to appear similar to official grocery store mobile application pages. It is the page with the most visual flair and acts as the "face" of the application, in a sense. The goal is to emulate the feel of perusing a well-organized catalogue of items, while providing helpful searching and sorting features that are effective for users of all skill levels. Every item also has an associated picture to improve visual identity. Searching and sorting features include a search bar at the top of the application, which can be used to search for grocery items by name. There is also the "search by Isle" feature, which will only display items in a particular category. Selecting an item through a search or sorting feature will bring you to that item's individual item page.
- **Development Process:** Development of the Item Catalogue main page interface has not been fully completed, and some aspects are subject to change. Item objects for the item catalogue interface are obtained by referencing the database. In particular, the `getitems()` method is used to reference item information from the database. Available catalogue grocery items are stored separately from list items through a "master list", which is strictly read-only. Every item has associated "identifier" values, such as its name,

category, and price. Every object also has an associated picture which acts as a visual representation for that object. A selection of items are chosen to appear on the main page based on the particular subsection. Selecting an item brings the user to the item's individual item page, where it may be added to the session cart. Current development has implemented the UI framework, associated individual item page screen builds, and search functionality. Full database integration, search by Isle, and item images have yet to be implemented.

Search Functionality

- **Conceptual Overview:** A search bar integrated at the top of the Item Catalogue main page screen build. Provides a key item search functionality to enhance the user experience and pinpoint particular items. The search takes a user input, and displays all items which contain that input (partial inputs are accepted). Of the Item Catalogue additional sorting functionalities, this feature is the most critical to implement and should be prioritized.
- **Development Process:** The Item Catalogue interface home screen implements an item search functionality. This is achieved through a traditional search bar widget (SearchBar). The search bar accepts a string of characters as input from the user. This string of characters is then compared against the name values of all grocery items in the database via instance object. Item names that contain the string of characters from the input are then displayed below the search bar. Multiple items that match the search criteria may be displayed at once, and interacting with one of these item names will move the user directly to that item's individual item page. This search bar is seamless, and automatically displays updated search items as the user changes their input in real-time through innate widget functionality. Specific search keywords may be implemented in a later build.

Item Details Page

- **Conceptual Overview:** This page is accessed after selecting an item in the main item catalogue. It displays a picture of the selected item front and center, with the items name and price in larger text below. The user may then select the quantity of the item they wish to add to the cart.
- **Development Process:** Upon screen instantiation, a function call takes the itemID from the selected item catalogue item catalogue and pulls its associated name and price values to draw the screen. The key functionality of this page is the add to cart widget. This widget allows the user to select the quantity of the item they would like, and press the add to cart button to add the item/items to their cart. This appends the selected item to their session cart and returns the user to the item catalogue homepage. Additional information,

such as ingredients and nutrition facts, are under consideration for inclusion on this page, but are not implemented in the current build.

Item Categorization

- **Conceptual Overview:** Similar items should have a category assigned to them for classification purposes. These categories will allow for more efficient item searching and sorting.
- **Development Process:** Every item is assigned a category value. This value is simply a logical identifier for what type of item this is. It is a data entry in each global items itemID document. For example, yogurt would be assigned the “diary” category value. The categories are designed to be easy to understand for all users. By using the “search by Isle” function, users can tighten their search to only include items from a specific category.

Item Recommendation

- **Conceptual Overview:** The Item Catalogue main page Interface has a subsection dedicated to recommending grocery items that the user might like. By interacting with the recommended items, users can immediately access them to add to their grocery list.
- **Development Process:** The recommended items are generated by referencing all items at the user has added to all their grocery lists in the database. When the screen is first built, a function call runs to select five items to be recommended here as items that the user may wish to add to their list. These five are obtained by crawling the groceryList item subcollections for this associated userID (in other words, the items in all of the users grocery lists). These item IDs are then used to read the five specific items in the Recommended Items section from the master list. The remaining five items are selected randomly from all available item objects in the master list. If the user does not have any list items in their current account, the recommended items are all completely random. This functionality has been designed, but has yet to be fully implemented.

Session Cart

- **Conceptual Overview:** This is the metaphorical “cart”, or in other words, the temporary list of items that the user wants to add to a specific grocery list, and is appended to a permanent grocery list via the checkout screen. Items selected through the item catalogue and individual item pages are added to the session cart.
- **Development Process:** The cart is represented by a stylized grocery cart button widget at the top of the screen. This button persists through all the item catalogue module’s sub-screens. The button has a small number next to it, representing the number of unique items in the cart. Interacting with this button will take the user to the checkout screen.

The button's functionality is handled by referencing a temporary list structure (this is a temporary internal data structure), which just holds the itemIDs for the item the user has selected in this session. This feature has not been fully implemented yet.

Checkout Screen

- **Conceptual Overview:** This is the page that is accessed upon clicking the session cart button from the main item catalogue interface. This page lists all of the items in the user's session cart, and at the bottom, provides the add-to-list button.
- **Development Process:** Interacting with the add-to-list button will append the items in the session cart to the specified list, and automatically return the user to that list's item screen in the list management interface. The add-to-list button is non-functional if there are no items in the session cart. The checkout screen displays the name and quantity of all items the user previously selected to add. This is handled by using the itemIDs in the temporary session lists to sequentially reference the corresponding item in the grocery master list.

Documentation - Item Database

Structure

Main Database

users (collection) - list of all users

└─ userId (document = Firebase UID) - each user is an individual document

name: "John"

email: john@email.com

groceryLists (collection) - all grocery lists

└─ listId (document = firebase auto-ID)

name: "Weekly"

ownerId: "john_uid"

members: ["userId_1", "userId_2", "userId_3", "userId_4"]

createdAt: timestamp

└─ items (subcollection)

└─ itemId (document = firebase auto-ID)

name: "Milk"

quantity: 1

completed: false

addedBy: "userId_3"

createdAt: timestamp

Global Items Catalog

catalog (collection)

└─ itemId (document = firebase auto-ID)

name: "Milk"

category: "Dairy"

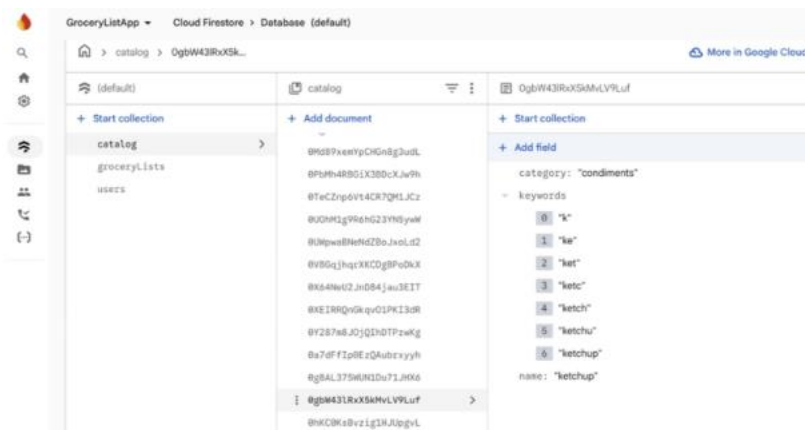
searchKeywords: ['m', 'mi', 'mil', 'milk', 'dairy']

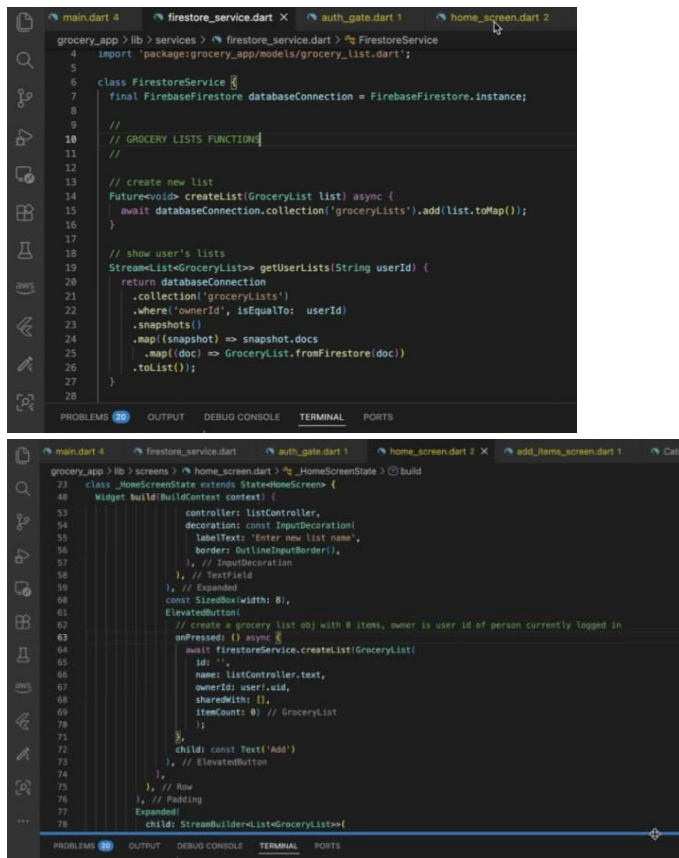
Database Connection

The database utilized for this application is the Firebase Real-Time Database, a cloud-based No SQL database that structures data as JSON files. Firebase permits real-time data synchronization across multiple connected clients, making it the ideal choice for implementing this app's real-time list synchronization functionality.

A Firebase connection is established by assigning a database connection function to a Firestone instance object. These artifacts are imported from the Firebase Options library specified in the code. The Firestone object acts as a traditional object which can run methods, with said methods being the main mechanism through which database operations are handled (such as reading and writing to the database).

For example, `Firestone.delete(ID)` would delete a specified object from the database. The database itself uses a nested structure of collections and documents.





Project Setup

The finalized build is a deployment-ready Flutter project that runs in the VScode environment with integrated Flutter and Dart extensions. Provided is a set of steps to run the project:

Prerequisites:

PC with Windows 10 or higher / Mac with MacOS 12.0+, Visual Studio Code (free version is fine), Git, Flutter VScode extension. Running the build natively on a phone requires a USB cable and an updated phone with developer mode enabled.

Installation Steps:

1. Download VScode and install the latest GIT package.
2. Open VScode and download the Flutter extension from the VScode marketplace.
3. Open the command palette with View > Command Palette and type in "Flutter".
4. Select Flutter > New Project and download the Flutter SDK when prompted.
5. Once the SDK is downloaded, a prompt will appear. Select Clone Flutter > Add SDK to path.
6. Run "Flutter Doctor" on the command to ensure installation was successful.

7. This should have also installed the Dart SDK. If not, the Dart extension may be downloaded individually from the VScode marketplace. Alternatively, you can run *\$ Dart install* on the command line interface.
8. You now have an empty Flutter project, access the current project build on gitHub and clone the repository here.
9. The project has now been fully set up. To deploy the application, select the run (green arrow button) at the top of the screen. When asked to configure a build, choose a browser like Google Chrome. The project can also be run natively on a mobile device if a USB cable is connected to the hosting computer.

Verification:

Upon a successful launch, the application will load the Grocery List Home Page UI in a web browser screen. This is essentially an emulation of the mobile screen view, allowing for easier testing (all functionality is maintained). This page consists of a username field, password field, register account button, and login button. All elements are interactable, and the user may move between and interact with all other implemented modules according to the specified functionality. Changes to the database are reflected in real-time. An unsuccessful launch will usually just result in a compilation error or blank window.

Test Plan

Purpose

The purpose of this document is to test and validate the software component modules of the Listin Grocery List application through individual component testing and integration testing, to ensure full application functionality and guide debugging. This document provides a set of test cases for the application and its associated database, in addition to all necessary context to define and facilitate testing. In accordance, this test plan will include:

- The scope of testing: what is in-scope and out-of-scope.
- Test cases for all application modules.
- Descriptions of context for all test scenarios.
- The expected results of all test scenarios.
- The execution steps of every test case.
- The software, hardware, and network environment needed for effective testing.
- Any required sample data necessary to facilitate test cases.
- A compiled Test Plan Report detailing the results of each test case.

Content to be Validated

Content to be Validated through testing is split between 6 key modules. These modules are Account Management, Grocery List Management, Item Catalogue, Database Structure, User Interface, and Shared List Updates. Content for integration test validation (testing interactions between multiple modules), is also included.

Account Management

- Ensure password registration validates against the correct password requirement conditions.
- Ensure account registration correctly writes credentials to Firebase.
- Ensure login succeeds using valid credentials.
- Ensure login fails using invalid credentials.
- Ensure log out ends the current session and locks further access.

Grocery List Management

- Ensure the add list button successfully writes the named list to Firebase.
- Ensure all lists correctly pull their associated grocery items from the Firebase.
- Stress test the grocery list homepage with a sufficiently high number of lists.
- Ensure list deletion via the delete button works successfully.
- Ensure grocery list item deletion works successfully.

Item Catalogue

- Ensure all catalogue items are pulled live from the database.
- Sufficiently test the *search bar functionality* to confirm responsive, name-based filtering of all items. Also test partial name queries.
- Ensure the *search by isle functionality* correctly filters items based on the specified grocery item isle type.
- Sufficiently test that the individual item page pulls the correct information on the selected catalogue item.
- Ensure the add to cart function successfully adds the specified item with the specified quantity to the session cart.

Database Structure

- Ensure database writes and all database content persist between different sessions.
- Ensure the three prime database structures: Users, Grocery Lists, and Item Catalogue, maintain the correct JSON substructure proposed by the back-end developer when content is added.

- Test speed of database writes and associated database pulls via StreamBuilder() to ensure the real-time update responsiveness.
- Ensure the Item Catalogue substructure cannot be modified by internal method calls, and only through direct developer database writes (security constraint).

User Interface

- Ensure all catalogue items correctly pull their associated image via the integrated image URLs.
- Ensure all interactive screen widgets load correctly when transitioning screens via the bottom menu.
- Ensure no duplicate grocery items with broken images appear in the item catalogue interface.

Shared List Updates

- Ensure other user accounts can be correctly identified when searching for an invite target on the account linking page.
- Confirm that only valid account email addresses receive the specified list editing invitation.
- Ensure all item additions and deletions performed on shared lists reflect immediately on the interfaces of all linked accounts.
- Ensure that multiple users (more than 2) can edit the list simultaneously without issue.

Integrated Testing

- Ensure that using add to list from the session cart writes the cart grocery items to the specified grocery list.
- Ensure all created grocery lists are selectable when prompted to choose a list from the session cart.
- Ensure the add to cart button from the grocery list homepage redirects immediately to the item catalogue.
- Ensure that the item quantity selected from the item catalogue is visible across all item-based interfaces.
- Perform a practical test run of the app from the perspective of a common user, from login to list creation, to logout, to ensure application validity.
- Ensure the application can run natively on both IOS and Android mobile devices.

Scope of Testing

In-Scope:

Modules/Features within the scope of testing:

- Account Management
- Grocery List Management
- Item catalogue

- Database Structure
- User Interface
- Shared List Updates
- Integrated Testing for multiple Modules

Out-of-Scope:

Modules/Features outside the scope of testing:

- Firestone service methods
- Firestone connection APIs
- Extraneous widget functionality
- Email verification functionality
- Item Catalogue Item Recommendation Feature

Test Cases

The following are the descriptions for each test case and their associated expected result for test validation. Tests are identified using a Test Case ID, which are structured as:

- Module to be Tested – Test Type – Test Number

Below is a key for identifying the shorthand used in the components of the Test Case ID:

Module	Test Type
ACCOUNT – Account Management	FUNC – Functional Testing
LIST – Grocery List Management	SEC – Security Testing
CAT – Item Catalogue	DATA – Database Testing
DB – Database Structure	STRESS – Stress Testing
UI – User Interface	MISC – Miscellaneous Testing
INT - Integrated	
SLU – Shared List Updates	

Note: All tests with the FUNC (functional) test ID are functional test scenarios. Tests without the FUNC test ID are non-functional test scenarios.

- Test Case ID: ACCOUNT-SEC-001
 - o Description: Password registration validates against the correct password requirement conditions.
 - o Expected Result: User password is accepted when the password conditions are met, and password is written to Firebase.
- Test Case ID: ACCOUNT-DATA-001
 - o Description: Account registration correctly writes credentials to Firebase.
 - o Expected Result: Specified Full Name, Email, and Password are written to the Users substructure in Firebase.
- Test Case ID: ACCOUNT-FUNC-001
 - o Description: Login succeeds using valid credentials.
 - o Expected Result: User enters valid credentials and is logged in to the grocery list management homepage.
- Test Case ID: ACCOUNT-SEC-002
 - o Description: Login fails using invalid credentials
 - o Expected Result: User enters invalid credentials, is not logged in, and a warning message is printed to screen.
- Test Case ID: ACCOUNT-FUNC-002
 - o Description: Log out ends the current session and locks further access.
 - o Expected Result: Users selects log out and is returned to an empty login page.
- Test Case ID: LIST-DATA-001
 - o Description: The add list button successfully writes the named list to Firebase.
 - o Expected Result: Users selects add list and the specified list is added to the Grocery List substructure in Firebase.
- Test Case ID: LIST-DATA-002
 - o Description: All lists correctly pull their associated grocery items from the Firebase.
 - o Expected Result: User opens multiple lists and their Grocery List items match the item add operations performed on specified lists.
- Test Case ID: LIST-STRESS-001
 - o Description: The grocery list homepage functions as intended with a sufficiently high number of lists.
 - o Expected Result: Users creates 30 grocery lists and can still operate the Grocery List Homepage as normal.
- Test Case ID: LIST-FUNC-001
 - o Description: List deletion via the delete button works successfully.
 - o Expected Result: User selects the trash can widget to delete a list, and it is immediately removed from the Grocery List homepage and Firebase.
- Test Case ID: LIST-FUNC-002
 - o Description: grocery list item deletion works successfully.

- o Expected Result: User selects the trash can widget to delete an item, and it is immediately removed from the Grocery List page and Firebase.
- Test Case ID: CAT-DATA-001
 - o Description: All catalogue items are pulled live from the database.
 - o Expected Result: Developer accesses the item catalogue Firebase substructure directly and adds a grocery item object, then the grocery item is added to a running instance of the item catalogue.
- Test Case ID: CAT-FUNC-001
 - o Description: Confirm the *search bar functionality* performs responsive, name-based filtering of all items, and works with partial name queries.
 - o Expected Result: User enters the partial names of multiple items, such as B A C, or C H I C, and the resulting displayed items are correct.
- Test Case ID: CAT-FUNC-002
 - o Description: Confirm the *search by isle functionality* correctly filters items based on the specified grocery item isle type.
 - o Expected Result: User selects search by isle and iterative checks all isles, and the filtered item contents all match their associated isle type (meat should only have items like bacon, turkey, and steak).
- Test Case ID: CAT-FUNC-003
 - o Description: Test that the individual item page pulls the correct information on the selected catalogue item
 - o Expected Result: User selects 10 individual item icons from the item catalogue and all 10 items have the correct name, image, and category on the individual item page.
- Test Case ID: CAT-FUNC-004
 - o Description: Add to Cart successfully adds the specified item with the specified quantity to the session cart.
 - o Expected Result: User selects three grocery items with different quantities, and all items and quantities match in the session cart after Add to Cart is selected.
- Test Case ID: DB-DATA-001
 - o Description: Database writes and all database content persist between different sessions.
 - o Expected Result: User creates an account and performs some standard list and item operations. User logs off and logs back in and all list and grocery item content is exactly the same.
- Test Case ID: DB-DATA-002
 - o Description: The three prime database structures: Users, Grocery Lists, and Item Catalogue, maintain the correct JSON substructure proposed by the back-end developer when content is added.
 - o Expected Result: User creates an account, grocery list, and adds items to the grocery list. Then, a developer confirms the data in the Firebase repository is structured as: Users > UserID, Grocery Lists > ListID > Items > ItemID, and Item Catalogue > ItemID. (devTest)
- Test Case ID: DB-FUNC-001

- o Description: Test the speed of database writes and associated database pulls via StreamBuilder() to ensure the real-time update responsiveness.
 - o Expected Result: User creates a list and the list is added to the Grocery List homepage in less than five seconds. User adds an item to a list and the associated grocery list is updated in less than five seconds.
- Test Case ID: DB-SEC-001
 - o Description: Security test to confirm Item Catalogue substructure cannot be modified by internal method calls, and only through direct developer database writes
 - o Expected Result: User cannot modify the Item Catalogue Firebase substructure in any way.
- Test Case ID: UI-FUNC-001
 - o Description: Ensure all catalogue items correctly pull their associated image via the integrated image URLs.
 - o Expected Result: All catalogue items have the correct image associated with their name when viewed from both the homepage and individual item page. No dead link URL images appear.
- Test Case ID: UI-FUNC-002
 - o Description: All interactive screen widgets load correctly when transitioning screens via the bottom menu.
 - o Expected Result: All module screen widgets load in their default state when rapidly switching between screens, even if the user is mid-operation in one module.
- Test Case ID: UI-MISC-001
 - o Description: Confirm no duplicate grocery items with broken images appear in the item catalogue interface (item duplication bug).
 - o Expected Result: User sees no duplicate grocery items with the same name when using the item catalogue homepage or search functionality.
- Test Case ID: INT-FUNC-001
 - o Description: Using add to list from the session cart writes the cart grocery items to the specified grocery list.
 - o Expected Result: User selects add to list with items in the session cart and selects a list from the dropdown menu, and the items are added to that list from the Grocery List homepage.
- Test Case ID: INT-FUNC-002
 - o Description: All created grocery lists are selectable when prompted to choose a list from the session cart.
 - o Expected Result: User creates multiple grocery lists and can select all of them from the session cart dropdown menu.
- Test Case ID: INT-FUNC-003
 - o Description: The add to cart button from the grocery list homepage redirects immediately to the item catalogue.
 - o Expected Result: User selects add to cart from the item subpage of a grocery list and the item catalogue screen is immediately loaded.
- Test Case ID: INT-FUNC-004

- o Description: The item quantity selected from the item catalogue is visible across all item-based interfaces.
 - o Expected Result: User adds an item from the item catalogue with a specified quantity, and the item quantity is visible and correct in both the session cart and grocery item subpage.
- Test Case ID: INT-FUNC-005
 - o Description: Perform a practical test run of the app from the perspective of a common user, from login > list creation > item selection > logout to ensure application validity.
 - o Expected Result: User successfully logs in, creates a list, adds items to the list, and logs out.
- Test Case ID: INT-MISC-001
 - o Description: Successfully run the application natively on IOS and Android mobile devices.
 - o Expected Result: App runs successfully on IOS and Android mobile phone.
- Test Case ID: SLU-FUNC-001
 - o Description: The account linking screen allows users to select one of their lists and send a request to another account to share the specified list.
 - o Expected Result: The account the list was sent to receives an invitation to work on the specified list on the account linking screen.
- Test Case ID: SLU-FUNC-002
 - o Description: Item additions to a shared list reflect immediately for all members of the shared list regardless of who initiates the addition.
 - o Expected Result: User adds an item to a shared list and all users immediately see the item that was added update in their version of the shared list.
- Test Case ID: SLU-FUNC-003
 - o Description: Item deletions to a shared list reflect immediately for all members of the shared list regardless of who initiates the deletion.
 - o Expected Result: User deletes an item from a shared list and all users immediately see the item that was deleted removed in their version of the shared list.
- Test Case ID: SLU-FUNC-004
 - o Description: The account linking supports more than 2 users being linked to the same shared list.
 - o Expected Result: User sends invites to two other users, and the shared list appears in all three user's grocery list management pages.
- Test Case ID: SLU-SEC-001
 - o Description: Shared list update invites may only be sent to accounts with a valid email address.
 - o Expected Result: User enters an invalid email address, and the invite is not sent and a warning message is printed.

Test Procedures

- Test Case ID: ACCOUNT-SEC-001
 1. Open App.

2. Select Register Account.
 3. Create a password that does not contain a special character.
 4. Register account.
 5. Registry fails and a warning message is printed.
 6. Enter password that contains a special character, uppercase character, lowercase character, and number.
 7. Register Account.
 8. Expected Result: User password is accepted when the password conditions are met, and password is written to Firebase.
- Test Case ID: ACCOUNT-DATA-001
 1. Open App.
 2. Select Register Account.
 3. Create Account that meets all registration requirements.
 4. Register Account.
 5. Check Firebase Users substructure.
 6. Expected Result: Specified Full Name, Email, and Password are written to the Users substructure in Firebase.
 - Test Case ID: ACCOUNT-FUNC-001
 1. Open App.
 2. Enter the credentials of an already registered account on the login screen.
 3. Select Log in.
 4. Expected Result: User enters valid credentials and is logged in to the grocery list management homepage.
 - Test Case ID: ACCOUNT-SEC-002
 1. Open App.
 2. Enter credentials that do not match any existing account.
 3. Select Log in.
 4. Expected Result: User enters invalid credentials, is not logged in, and a warning message is printed to screen.
 - Test Case ID: ACCOUNT-FUNC-002
 1. Open App.
 2. Log in.
 3. Move to the Settings screen.
 4. Select Log off.
 5. Expected Result: User selects log out and is returned to an empty login page.
 - Test Case ID: LIST-DATA-001
 1. Open App.
 2. Log in.
 3. Enter a name in the Grocery List add name text box.
 4. Select Add List.

5. Check Firebase Grocery Lists subsection.
 6. Expected Result: User selects add list and the specified list is added to the Grocery List substructure in Firebase.
- Test Case ID: LIST-DATA-002
 1. Open App.
 2. Log in.
 3. Create two grocery lists.
 4. Add different items to each of the two grocery lists.
 5. Expected Result: User opens multiple lists and their Grocery List items match the item add operations performed on specified lists.
 - Test Case ID: LIST-STRESS-001
 1. Open App.
 2. Log in.
 3. Create 30 grocery lists in quick succession.
 4. Open five grocery lists and quickly return to the homepage.
 5. Add items to one of the grocery lists.
 6. Open grocery list and confirm items were added.
 7. Expected Result: User creates 30 grocery lists and can still operate the Grocery List Homepage as normal.
 - Test Case ID: LIST-FUNC-001
 1. Open App.
 2. Log in.
 3. Create a grocery list.
 4. Select the red trash can icon next to the created grocery list.
 5. Confirm the list disappears from the homepage.
 6. Check the Firebase Grocery List substructure to confirm it does not exist.
 7. Expected Result: User selects the trash can widget to delete a list, and it is immediately removed from the Grocery List homepage and Firebase.
 - Test Case ID: LIST-FUNC-002
 1. Open App.
 2. Log in.
 3. Create a grocery list.
 4. Add items to the created grocery list.
 5. Select the red trash can next to an item.
 6. Confirm the item is removed from the items subpage.
 7. Check the Firebase ListID > Items substructure to confirm the deletion of the item.
 8. Expected Result: User selects the trash can widget to delete an item, and it is immediately removed from the Grocery List page and Firebase.
 - Test Case ID: CAT-DATA-001

1. Log in to Firebase.
 2. Open the Item Catalogue repository.
 3. Create a grocery item object.
 4. Assign a name.
 5. Assign a type.
 6. Assign a price.
 7. Open App.
 8. Log in.
 9. Move to Item Catalogue.
 10. Confirm the added item exists.
 11. Expected Result: Developer accesses the item catalogue Firebase substructure directly and adds a grocery item object, then the grocery item is added to a running instance of the item catalogue.
- Test Case ID: CAT-FUNC-001
 1. Open App.
 2. Log in.
 3. Move to Item Catalogue.
 4. Select Search Bar.
 5. Type a full product name (such as “Baking Powder”).
 6. Confirm search item name is the only item result.
 7. Type a partial product name (such as “yogu”).
 8. Confirm items that include the partial name are the only item results.
 9. Expected Result: User enters the partial names of multiple items, such as B A C, or C H I C, and the resulting displayed items are correct.
 - Test Case ID: CAT-FUNC-002
 1. Open App.
 2. Log in.
 3. Move to item catalogue.
 4. Select “Search by Isle”.
 5. Select Meat.
 6. Check all items match the meat label.
 7. Repeat for all 10 item categories.
 8. Expected Result: User selects search by isle and iterative checks all isles, and the filtered item contents all match their associated isle type (meat should only have items like bacon, turkey, and steak).
 - Test Case ID: CAT-FUNC-003
 1. Open App.
 2. Log in.
 3. Move to Item Catalogue.
 4. Select an item.

5. Confirm name, type and price of the item match on the individual item page.
 6. Repeat for 10 different items.
 7. Expected Result: User selects 10 individual item icons from the item catalogue, and all 10 items have the correct name, image, and category on the individual item page.
- Test Case ID: CAT-FUNC-004
 1. Open app.
 2. Log in.
 3. Move to Item Catalogue.
 4. Select three different items, and set each item a unique quantity.
 5. Select “Add to Cart” for each item.
 6. Move to the session cart.
 7. Confirm that all three items and their associated quantities match the item catalogue.
 8. Expected Result: User selects three grocery items with different quantities, and all items and quantities match in the session cart after Add to Cart is selected.
 - Test Case ID: DB-DATA-001
 1. Open App.
 2. Log In.
 3. Create a grocery list.
 4. Add items to the grocery list.
 5. Log off.
 6. Log back in.
 7. Confirm the created list and items added to the list are the same.
 8. Expected Result: User creates an account and performs some standard list and item operations. User logs off and logs back in and all list and grocery item content is exactly the same.
 - Test Case ID: DB-DATA-002
 1. Open App.
 2. Create an account.
 3. Log in.
 4. Create grocery list.
 5. Add items to grocery list.
 6. Open Firebase repository.
 7. Check new account is structured Users > UserID.
 8. Check that new list and items are structured Grocery Lists > ListID > Items > ItemID.
 9. Expected Result: User creates an account, grocery list, and adds items to the grocery list. Then, a developer confirms the data in the Firebase repository is

structured as: Users > UserID, Grocery Lists > ListID > Items > ItemID, and Item Catalogue > ItemID (DevTest).

- Test Case ID: DB-FUNC-001
 1. Open App.
 2. Log in.
 3. Open Firebase repository simultaneously.
 4. Create a grocery list in the application.
 5. Simultaneously check that the grocery list is added to both the Firebase repository and grocery list homepage in less than five seconds.
 6. Perform the same operations for adding an item to a grocery list through the session cart.
 7. Simultaneously check that the grocery list is added to both the Firebase repository and grocery list item page in less than five seconds.
 8. Expected Result: User creates a list and the list is added to the Grocery List homepage in less than five seconds. User adds an item to a list and the associated grocery list is updated in less than five seconds.
- Test Case ID: DB-SEC-001
 1. Open the Integrated Development Environment.
 2. Use the VSsearch functionality to check all Firestone service objects.
 3. Confirm no Firestone service methods take the Item Catalogue repository for a Send() argument.
 4. Expected Result: User cannot modify the Item Catalogue Firebase substructure in any way.
- Test Case ID: UI-FUNC-001
 1. Open App.
 2. Log in.
 3. Move to Item Catalogue
 4. Check all items by category using “Search by Isle”.
 5. Confirm that all items have the correct associated picture.
 6. Select 10 items at random and check their individual item pages.
 7. Confirm that the individual item page images match the item catalogue images.
 8. Expected Result: All catalogue items have the correct image associated with their name when viewed from both the homepage and individual item page. No dead link URL images appear.
- Test Case ID: UI-FUNC-002
 1. Open App.
 2. Log in.
 3. Rapidly switch between screens using the bottom screen interface.
 4. Confirm that all interactive widget buttons load instantly and correctly when switching screens. Perform 10 or more times to reinforce testing.

5. Add an item to the session cart.
 6. Open the menu to select the list to add too.
 7. While mid-operation, switch to a different screen.
 8. Return to the session cart screen.
 9. Expected Result: All module screen widgets load in their default state when rapidly switching between screens, even if the user is mid-operation in one module.
- Test Case ID: UI-MISC-001
 1. Open app.
 2. Log in.
 3. Move to Item Catalogue.
 4. Search items by aisle.
 5. Confirm that there are no duplicate items with the exact same name and image (some items have the same image).
 6. Open Firebase repository.
 7. Confirm no Item Catalogue objects have the exact same ItemID.
 8. Expected Result: User sees no duplicate grocery items with the same name when using the item catalogue homepage or search functionality.
 - Test Case ID: INT-FUNC-001
 1. Open app.
 2. Log in.
 3. Create two empty grocery lists.
 4. Add items to the session cart.
 5. Select Add to List.
 6. In the dropdown menu, select the second list.
 7. Confirm the items are added exclusively to the second list.
 8. Expected Result: User selects add to list with items in the session cart and selects a list from the dropdown menu, and the items are added to that list from the Grocery List homepage.
 - Test Case ID: INT-FUNC-002
 1. Open app.
 2. Log in.
 3. Create multiple grocery lists.
 4. Add items to the session cart.
 5. Select Add to List.
 6. When prompted to choose a list to add to, confirm that all created lists are available options in the dropdown menu.
 7. Expected Result: User creates multiple grocery lists and can select all of them from the session cart dropdown menu.
 - Test Case ID: INT-FUNC-003

1. Open App.
 2. Log in.
 3. Create a Grocery List.
 4. Add items to the session cart.
 5. Select Add to List.
 6. Select the created list from the dropdown menu.
 7. Confirm screen view automatically transitions to the grocery list item page of the newly added to list.
 8. Expected Result: User selects add to cart from the item subpage of a grocery list and the item catalogue screen is immediately loaded.
- Test Case ID: INT-FUNC-004
 1. Open App.
 2. Log in.
 3. Create an empty grocery list.
 4. Move to item catalogue.
 5. Select an item.
 6. Assign the item a quantity and select “Add to Cart”.
 7. Move to session cart.
 8. Confirm the item’s quantity matches the set quantity.
 9. Add the item to the created grocery list.
 10. Open the grocery list.
 11. Confirm the item’s quantity in the grocery list matches the set quantity.
 12. Expected Result: User adds an item from the item catalogue with a specified quantity, and the item quantity is visible and correct in both the session cart and grocery item subpage.
 - Test Case ID: INT-FUNC-005
 1. Open app.
 2. Create account.
 3. Log in.
 4. Create grocery list.
 5. Move to Item Catalogue.
 6. Select an item.
 7. Select “Add to Cart”.
 8. Move to session cart.
 9. Select “Add to List”.
 10. Select the created list.
 11. Confirm items are added to the list.
 12. Move to settings page.
 13. Select Log off.

14. Expected Result: User successfully logs in, creates a list, adds items to the list, and logs out.
- Test Case ID: INT-MISC-001
 1. Connect phone to application hosting computer via USB.
 2. On phone, open settings.
 3. Select “enable developer mode” in settings.
 4. Select “run” from the integrated development environment.
 5. When prompted to choose a runtime environment, the model of the connected phone should appear.
 6. Select “run from (iPhone/Android)”.
 7. Confirm app launches on the physical mobile device.
 8. Perform standard application operations such as logging in, creating a list, and adding items, to test validity.
 9. Expected Result: App runs successfully on IOS and Android mobile phones.
 - Test Case ID: SLU-FUNC-001
 1. Open app.
 2. Log in.
 3. Create a grocery list.
 4. Move to account linking screen.
 5. Select “send invite”.
 6. Select the created grocery list.
 7. Enter the email address of another valid email account.
 8. Select send.
 9. Log out.
 10. Log in to the account the invite was sent to.
 11. Expected Result: The account the list was sent to receives an invitation to work on the specified list on the account linking screen.
 - Test Case ID: SLU-FUNC-002
 1. Open app.
 2. Log in.
 3. Create a grocery list.
 4. Move to account linking screen.
 5. Select “send invite”.
 6. Select the created grocery list.
 7. Enter the email address of another valid email account.
 8. Select send.
 9. Simultaneously open the recipient account.
 10. Accept the invite.
 11. Add an item to the shared grocery list.

12. Expected Result: User adds an item to a shared list and all users immediately see the item that was added update in their version of the shared list.
- Test Case ID: SLU-FUNC-003
 1. Open app.
 2. Log in.
 3. Create a grocery list.
 4. Add an item to the grocery list.
 5. Move to account linking screen.
 6. Select “send invite”.
 7. Select the created grocery list.
 8. Enter the email address of another valid email account.
 9. Select send.
 10. Simultaneously open the recipient account.
 11. Accept the invite.
 12. Delete an item from the shared grocery list.
 13. Expected Result: User deletes an item from a shared list and all users immediately see the item that was deleted removed in their version of the shared list.
 - Test Case ID: SLU-FUNC-004
 1. Open app.
 2. Log in.
 3. Create a grocery list.
 4. Move to account linking screen.
 5. Select “send invite”.
 6. Select the created grocery list.
 7. Enter the email address of another valid email account.
 8. Select send.
 9. Repeat steps 5-8, but with a different account as the recipient.
 10. Simultaneously open both recipient accounts (three accounts open).
 11. Accept the invite on both recipient accounts.
 12. Expected Result: User sends invites to two other users, and the shared list appears in all three user’s grocery list management pages.
 - Test Case ID: SLU-SEC-001
 1. Open app.
 2. Log in.
 3. Create a grocery list.
 4. Move to account linking screen.
 5. Select “send invite”.
 6. Select the created grocery list.
 7. Enter an invalid email address into the recipient field.
 8. Select send.

9. Expected Result: User enters an invalid email address, and the invite is not sent and a warning message is printed.

Test Environment

Hardware Environment:

Hardware Environment needed for testing

- Processor with minimum 2 GHz speed + 64-bit architecture
- 2-3 GB of RAM
- Adequate phone storage / disk space (Estimated size around 100-300 MB. App size is estimated to be around this level, or slightly smaller)
- Minimum screen resolution to properly render graphics / UI
- Functional touch screen

Software Environment:

Software Environment needed for testing

- Latest Windows or MacOS version with Flutter support (Windows 10 has the highest compatibility).
- The Visual Studio Code source code editor.
- The Visual Studio Code Flutter Extension.
- The Visual Studio Code Dart Extension.
- GIT version 2.53.0 or later (A *git install* through the terminal may be necessary).
- The built-in Flutter virtual device emulator for running test on the mobile application.

Network Setup:

Network Setup needed for testing

- All Firestone service APIs for connecting to the database (inherited from the Firebase library).
- Direct developer access credentials to the Grocery App Firebase data repository (necessary to confirm database writes).
- A working internet connection.

Required Test Data

In terms of required data for testing, a good amount of material is necessary to fully test the application. The following measures are estimates and may not exactly reflect the exact amount of data needed.

Required Data	Amount
Full project codebase	1
All imported Firebase libraries	3
Sample user accounts	5
Grocery List JSON files	15
Grocery Item JSON files	100
Firebase data repository	1

Test Report

Test Case	Pass	Fail	Severity
ACCOUNT-SEC-001 Password registration validates against the correct password requirement conditions.	YES		Moderate
ACCOUNT-DATA-001 Account registration correctly writes credentials to Firebase.	YES		Critical
ACCOUNT-FUNC-001 Login succeeds using valid credentials.	YES		Critical
ACCOUNT-SEC-002 Login fails using invalid credentials	YES		Critical
ACCOUNT-FUNC-002 Log out ends the current session and locks further access.	YES		Critical
LIST-DATA-001 The add list button successfully writes the named list to Firebase.	YES		Critical
LIST-DATA-002 All lists correctly pull their associated grocery items from the Firebase.	YES		Critical

LIST-STRESS-001 The grocery list homepage functions as intended with a sufficiently high number of lists.	YES		Moderate
LIST-FUNC-001 List deletion via the delete button works successfully.	YES		Critical
LIST-FUNC-002 Grocery list item deletion works successfully.	YES		Critical
CAT-DATA-001 All catalogue items are pulled live from the database.	YES		Critical
CAT-FUNC-001 Confirm the <i>search bar functionality</i> performs responsive, name-based filtering of all items, and works with partial name queries.	YES		Moderate
CAT-FUNC-002 Confirm the <i>search by isle functionality</i> correctly filters items based on the specified grocery item isle type.	YES		Moderate
CAT-FUNC-003 Test that the individual item page pulls the correct information on the selected catalogue item	YES		Critical
CAT-FUNC-004 Add to Cart successfully adds the specified item with the specified quantity to the session cart.	YES		Critical
DB-DATA-001 Database writes and all database content persist between different sessions.	YES		Critical

DB-DATA-002 The three prime database structures: Users, Grocery Lists, and Item Catalogue, maintain the correct JSON substructure proposed by the back-end developer when content is added.	YES		Critical
DB-FUNC-001 Test the speed of database writes and associated database pulls via StreamBuilder() to ensure the real-time update responsiveness.	YES		Moderate
DB-SEC-001 Security test to confirm Item Catalogue substructure cannot be modified by internal method calls, and only through direct developer database writes	YES		Critical
UI-FUNC-001 Ensure all catalogue items correctly pull their associated image via the integrated image URLs.	YES		Moderate
UI-FUNC-002 All interactive screen widgets load correctly when transitioning screens via the bottom menu.	YES		Moderate
UI-MISC-001 Confirm no duplicate grocery items with broken images appear in the item catalogue interface (item duplication bug).	YES		Moderate
INT-FUNC-001 Using add to list from the session cart writes the cart grocery items to the specified grocery list.	YES		Critical

INT-FUNC-002 All created grocery lists are selectable when prompted to choose a list from the session cart.	YES		Critical
INT-FUNC-003 The add to cart button from the grocery list homepage redirects immediately to the item catalogue.	YES		Minor
INT-FUNC-004 The item quantity selected from the item catalogue is visible across all item-based interfaces.	YES		Moderate
INT-FUNC-005 Perform a practical test run of the app from the perspective of a common user, from login > list creation > item selection > logout to ensure application validity.	YES		Critical
INT-MISC-001 Successfully run the application natively on IOS and Android mobile devices.	YES		Critical
SLU-FUNC-001 The account linking screen allows users to select one of their lists and send a request to another account to share the specified list.	YES		Critical
SLU-FUNC-002 Item additions to a shared list reflect immediately for all members of the shared list regardless of who initiates the addition.	YES		Critical
SLU-FUNC-003	YES		Critical

Item deletions to a shared list reflect immediately for all members of the shared list regardless of who initiates the deletion.			
SLU-FUNC-004 The account linking supports more than 2 users being linked to the same shared list.	YES		Moderate
SLU-SEC-001 Shared list update invites may only be sent to accounts with a valid email address.	YES		Moderate

Version Control

Strategy:

This project utilizes GitHub and Git to manage version control for the development team. All source code and associated APIs are integrated with complete version control based in a central GitHub repository. Version control is vital to our project, as the development process is split between two specialized developers: Jason, who focused on front-end development, and Amaya who focused on back-end development and database integration. At the beginning of development, each developer forked a branch to individually implement and test their associated specialized modules. For example, Jason would work on the item catalogue while Amaya would work on the Grocery List database integration. These updates were version controlled through regular commits to their personal branches. At certain intervals throughout the project, the personal repositories would be jointly analyzed and merged to create a full build. This would then act as the base for the next round of development. Developers coordinated in advance when commits would occur to avoid potential conflicts.

Repository Structure

- Main – Initial repository which is updated with the latest fully integrated build.
- Amaya-work – Core branch for back-end feature development and database integration.
- Jason-work – Core branch for front-end feature development.

Directory Organization

This is a general model of the project directories. The critical directories are expanded to showcase the organization and structure. The other directories are mostly for handling launches on different platforms or testing.

- android/
- ios/
- Lib/
 - auth/
 - auth_gate.dart
 - models/
 - cart_item.dart
 - catalog_item.dart
 - grocery_item.dart
 - grocery_list.dart
 - list_invite.dart
 - user_model.dart
 - screens/
 - CatalogScreen.dart
 - account_linking_screen.dart
 - account_screen.dart
 - aisle_results_screen.dart
 - aisle_screen.dart
 - cart_screen.dart
 - firebase_options.dart
 - home_screen.dart
 - item_detail_screen.dart
 - login_screen.dart
 - register_screen.dart
 - show_items_screen.dart
 - welcome_screen.dart
 - services/
 - firestore_service.dart
 - populate_catalog.dart
 - firebase_options.dart
 - main.dart
- linux/
- macos/
- test/
- web/
- windows/
- gitignore
- metadata

Commit History

Version control includes a comprehensive commit history which tracks the following:

- Initial project setup with Flutter and basic.
- Login, register, and home screens.
- Add items and Item Catalogue.
- Firestone services.

- Database connectivity.
- Database Structure Changes.
- UI updates.
- Account Linking.
- Cart System.
- Testing and bug fixes.
- Documentation and Notes

Example:

Commits on Mar 26, 2026		
updated catalog screen ui	jasonksu committed last month	a586e63  <>
implement cart system, account/logout screen, and Firebase-powered catalog and grocery lists	jasonksu committed last month	b406b70  <>
Commits on Mar 25, 2026		
Merge branch 'main' of https://github.com/Senior-Project-Blue-Grocery/grocery-list-mobile-app	jasonksu committed last month	112005f  <>
Connect dabase to catalogue	jasonksu committed last month	f4ebc4e  <>
Commits on Mar 21, 2026		
modified home and add items screen to use firestore service functions to edit database	amaya1182 committed last month	f4eb25e  <>
Commits on Mar 18, 2026		
added seed function to fill the item catalog	amaya1182 committed on Mar 18	aaeb065  <>
Commits on Mar 7, 2026		
added firestore service file with functions for adding to database	amaya1182 committed on Mar 7	a221479  <>

Summary

Overall, we feel that *Listin* has met the goals defined in the SP-5 project prompt. We have provided a comprehensive overview of our full Software Development Life Cycle. After successful planning, requirements specification, and software design definition phases, development began in earnest. Development continued for roughly three months, with the key modules being developed and integrated at a steady pace.

Account Management and Grocery List Management were finished first, then the Item Catalogue, User Interface, and Database Structure, then finally the Shared List Updates. The cross-

account list updates were the last key feature to be implemented. Throughout the development process, Firestone service methods were used to connect the application to the Firebase online repository through APIs. This served to achieve the collaborative real-time update functionality which is critical to the app. As development continued, all component functionalities and their implementations were recorded in the development document, which was completed alongside the application. After initial development concluded, testing was carried out on the prototype build using a module-based approach.

Each module was thoroughly tested according to the specified functional and nonfunctional test cases, then integrated testing was conducted. Initial testing results were highly promising, which the majority of critical test cases passing. This was likely due to the highly stable prototype build, which was fully integrated with Firebase at this point. The testing results were compiled into a Software Test Report. Finally, another round of debugging was conducted, and the finished deployment-ready project build was completed. The project used Github with three major branches for version control. The main branch, with the integrated milestone builds, and two sub-branches, where front-end and back-end features were developed and tested before merging with the main branch.

Now that development has been completed, we can take the chance to look back at our final product. We feel that *Listin* fulfills our vision of a convenient, user-friendly collaborative grocery app. The app allows users to intuitively create and edit grocery lists, select items from an extremely detailed item catalogue, and link friends and family to share the same lists. We placed extra emphasis on the UI and real-time update feature to ensure the app is extremely convenient to use while still achieving the required functionality. The hardworking development team, strong screen mock-up designs, and early development environment choices to use Flutter/Dart and Firebase led to a smooth Development Cycle. Our final result is a fully realized mobile app capable of running natively on IOS and Android which provides a uniquely convenient way to manage grocery lists. We are proud of our project and believe it achieves all that we set out for, both in terms of our own vision and the project requirements.

Appendix

Project Plan

Project Overview/Abstract

The SP-5 Grocery List App is a mobile application that supports collaborative grocery list management using shared groups. The app allows users to create accounts, share accounts, and individually read, update, delete, and create grocery lists. An important requirement is to provide real-time data synchronization. This would essentially allow any data modification performed by the members of the group to reflect in real time on all other smart devices. Additionally, the app needs to have backend features such as user authentication, access control

based on the group, data access by multiple users at the same time, etc. The scope encompasses aspects such as communicating a mobile client-server, real-time database integration, and multi-user communication in a networked environment.

Deliverables Specific to Project

1. A completed grocery list mobile application created using the Flutter or React Native framework. Grocery list mobile application should be fully functional, and include the following features: create accounts, add members to accounts/grocery lists, add and remove items from grocery list. In addition, updates to the grocery list should be reflected in real time.
2. Grocery Store mobile application source code.
3. Copy/list of raw data for all available grocery store items (could include third-party data)
4. Prototype application screen mockups created in a tool such as Figma by all members.
5. Final report package, which includes evidence all members completed the tutorial for flutter or react native. In addition, this should contain the final report, source code, a website with a link to a group GitHub, and Final Presentation Video.
6. Software design document to define implementation of the mobile application. Must also include group screen mockups to show information flow.
7. Development document to maintain a record of the grocery list application and its phases throughout the software development cycle.
8. Software requirements specification to define the functional requirements of the project. (common deliverable)
9. Test Plan and Report for testing project (common deliverable)
10. Prototype Presentation (common deliverable)

Group Meeting Times

- Weekly, Tuesday and Thursday, 1:45-3:30

Collaboration and Communication Plan

- Live Meeting in the KSU Marietta Library Twice Weekly
- Simple online communication handled through text group chat
- Document/content hosting and collaborative work handled through Microsoft Teams
- Online meetings, when necessary, are handled through Microsoft Teams
- A shared github account is used for the project

Project Schedule and Task Planning

The SP-5 Grocery List App project schedule is determined based on the Project Work Plan (Gantt chart) attached to the project to organize the activities based on the key stages of Software Development Life Cycle (SDLC). These are the Requirements Analysis, Design, Implementation, Testing, and Documentation/Deployment and are obtained to the deliverables of

the project group assignment. Within the Requirements Analysis, the team will assess the scope of the project, identify functional and non-functional requirements, choose the development framework, go through the necessary tutorials, and create screen mockups on an individual basis. The Design phase documents system architecture, real-time data synchronization plans, user interface flow, and Software Design Document. The implementation involves the development of authentication systems, shared accounts, building and maintaining grocery lists, and keeping real-time data and using GitHub as a version control tool. During the Testing phase, functional correctness, multi-user synchronization and system reliability are validated, and Test Plan and Report are generated. The Documentation and Deployment stage assembles the final report package, packages the source repository and the project site, and makes the final presentation materials. The Gantt chart displays the approximate durations of the tasks to be completed on the first planning level and is constantly updated in order to record the actual hours spent on the task and revise the schedule during the entire project lifecycle.

Other Plans

Competitive Analysis/Benchmarking - Many grocery businesses run successful large scale grocery mobile applications. I have used some of these applications in my own everyday life for grocery shopping. We should perform a competitive analysis of these applications to generate useful frameworks and concepts for developing a successful grocery mobile application. This could also help us define a more realistic scope for the project and provide inspiration for UI/UX design. Examples include the Publix and Kroger mobile apps.

Version Control Plan

As we are developing a mobile application with a group, version control is important for organizing development. We have decided to use Git, an open-source distributed version control system for our version control plan. Our codebase is hosted using a github repository, which is used to structure the project and track project history. This will allow our multiple developers to work on the project remotely. We will facilitate effective version control by maintaining the history of changes. We will also use effective communication and developer comments to ensure this. Our goal is standard and effective version control.

Gantt Chart

Project Name:		SP-5 Gcorcity (Blue)															
		Milestone #1				Milestone #2				Milestone #3				C-Day			
Phase	Tasks	Complete%	1-Feb	8-Feb	15-Feb	22-Feb	1-Mar	8-Mar	15-Mar	22-Mar	29-Mar	5-Apr	12-Apr	19-Apr	26-Apr	3-May	
Project Planning	Braintstorm Project	100%	3.5	2	0	0	0	0	0	0	0	0	0	0	0	0	
	Initial Planning	100%	3.5	0	0	0	0	0	0	0	0	0	0	0	0	0	
	Project Selection	100%	0	2.5	0	0	0	0	0	0	0	0	0	0	0	0	
	Project Plan	100%	0	0.5	1.5	0	0	0	0	0	0	0	0	0	0	0	
Design/Pre-Dev	Define Requirements	100%	3	0	0	0	0	0	0	0	0	0	0	0	0	0	
	Define Software Design	100%	0	2	2	0	0	0	0	0	0	0	0	0	0	0	
	Create SRS	100%	0	3	1	0	0	0	0	0	0	0	0	0	0	0	
	Create SDD	100%	0	3	3	0	0	0	0	0	0	0	0	0	0	0	
	Create Screen Mockups (Figma)	100%	0.5	2.5	0	0	0	0	0	0	0	0	0	0	0	0	
	Study Flutter and Dart / Complete Tutorial	100%	2	1	1	0	0	0	0	0	0	0	0	0	0	0	
Prototype Development	Develop Prototype Front-End	100%	1	1	2	2	0	1	1.5	1.5	1	1	2	1	1	0	
	Develop Prototype Back-End	100%	3.5	3	4	2	1	3	4	4	3.5	2	2	0	1	0	
	Create Development Document	100%	0	0	2	3	3	3	4	0	0	0	0	0	0	0	
	Create and Practice Prototype Presentation	100%	0	0	0	1	0	0	2	2	4	4	0	0	0	0	
	Peer Reviews + Advice Implementation	100%	0	0	0	0	1	1	1	4	5	5	4	0	0	0	
	Test and Debug Prototype	100%	0	0	0	0	1	1	3	2	2	3	1	2	2	0	
Final Phase / Miscellaneous	Maintain Documentation	100%	0	0	1	1	0.5	1.5	1	1	1	1	1	1	0	0	
	Additional Testing/Bug Fixes	100%	0	0	0	0.5	0.5	0.5	0.5	0.5	1	1	1	2	1	0	
	Create Final Report Document	100%	0	0	0	0	1	1	0	1	2	1	4	3	0	0	
	Create Final Report Video	10%	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
	Create Final Report Project Website	10%	0	0	0	0	0	0	0	0	0	0	0	2	0	0	
			17	20.5	17.5	9.5	8	12	17	16	19.5	18	15	11	5	0	
Team Total:			186														

Flutter Tutorial Completion

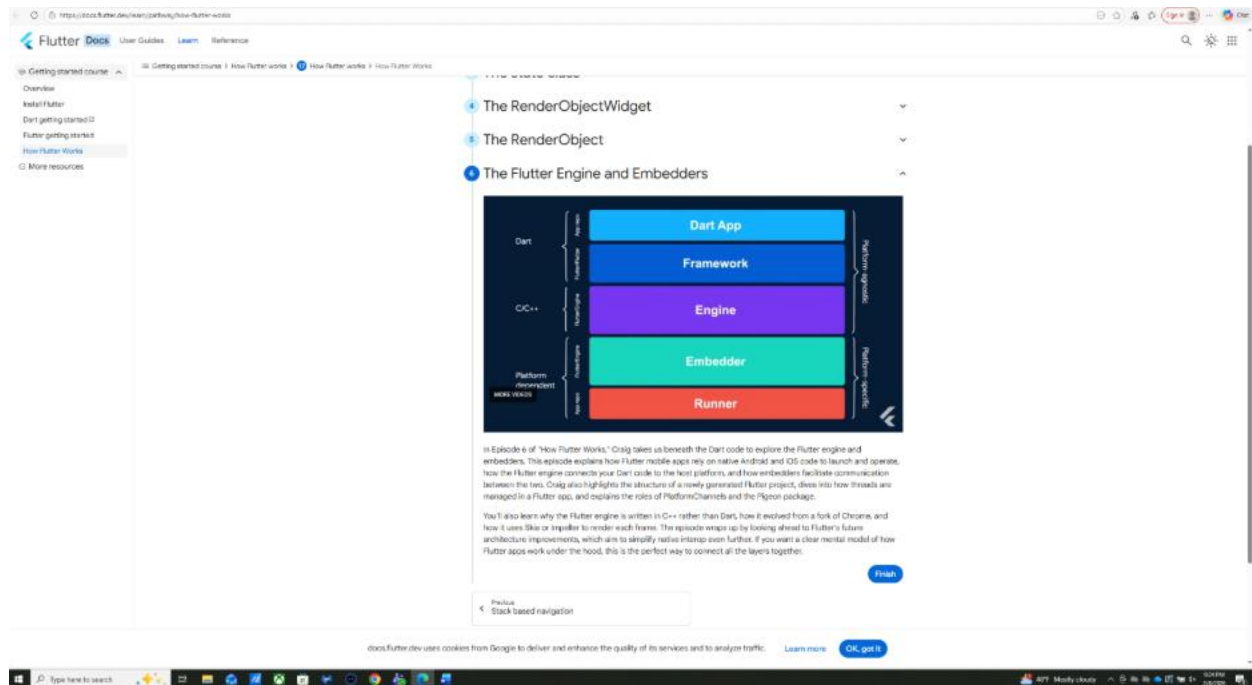
Our group has completed the following Flutter tutorial for learning purposes. A screenshot of the tutorial is provided below to show its group use:

Jack Lusk

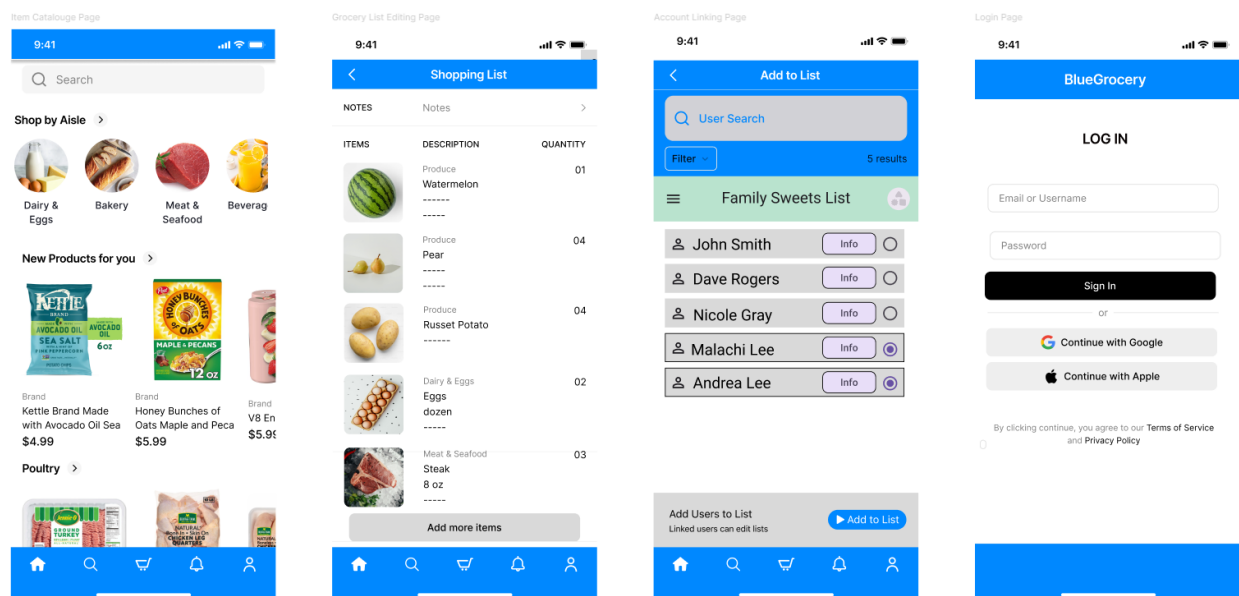
Jason Appolon

Amaya McCullough

Mahliq Obie



Figma Screen Mockups



Works Cited

Guide to app architecture. (2025). Flutter.dev. <https://docs.flutter.dev/app-architecture/guide>